数学与 CS 问答笔记

answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

November 12, 2026

Contents

1. 两种 spec 写法有什么区别：JSDoc 的 `@param` 与 requires/effects 三段式 6
   1.1. 费曼直觉 6
   1.2. 硬核机制 6
      1.2.1. 先拆语义，再谈语法 6
      1.2.2. 为什么第一种把后置条件写进 `@param` 6
      1.2.3. 关键纠偏：这两种都是声明式，不是「声明式 vs 操作式」 7
      1.2.4. 互译：同一份契约的两种载体 7
      1.2.5. Go 里没有这两种标签，合同写在哪 8
      1.2.6. 映射回 `internal/db`：先分类，再填栏 9
   1.3. 陷阱与易错 9
   1.4. 关键点 10
   1.5. 变式检验 10
   1.6. 同类归类 11
   1.7. Go 语言机制补充 11
   1.8. 本次操作与当前状态 12
2. Go 的 spec 怎么写：标准库与 Google 开源项目的真实样本 13
   2.1. 费曼直觉 13
   2.2. 硬核机制 13
      2.2.1. 规则先行：Google Go Style Guide 的四条硬要求 13
      2.2.2. 样本一：接口级合同，`io.Reader` 13
      2.2.3. 样本二：使用约束，`sync.Mutex` 14
      2.2.4. 样本三：同名不同合同，`os.File.Close` 与 `sql.Rows.Close` 15
      2.2.5. 样本四：并发安全，`net/http.Client` 16
      2.2.6. 样本五：前置条件的两条路线，`proto.Unmarshal` 与 `json.Unmarshal` 16
      2.2.7. 样本六：明确写出「不承诺什么」，`cmp.Diff` 17
      2.2.8. 样本七：错误类型必须写，`os.Chdir` 17
      2.2.9. 样本八：反例，变异没写进合同 17
      2.2.10. 汇总：八个样本覆盖哪一栏 18
      2.2.11. 映射回 `internal/db`：五个方法该参照谁 18
   2.3. 陷阱与易错 19
   2.4. 关键点 19
   2.5. 变式检验 20
   2.6. 同类归类 20
   2.7. Go 语言机制补充 20
   2.8. 本次操作与当前状态 21
3. Get 的 spec 首轮评审：一条假前置条件与两处缺条款 23
   3.1. 费曼直觉 23
   3.2. 硬核机制 23
      3.2.1. 逐句核对 23
      3.2.2. 实测：四个探针 23
      3.2.3. P0 一：前置条件没有被执行 24

3.2.4. P0 二：`val == nil` 不能当作未命中判据 25
3.2.5. P0 三：没写返回值是否别名 rep 25
3.2.6. P1 三项 25
3.2.7. 与测试的一致性 25
3.2.8. 真实项目对照 25
3.3. 陷阱与易错 26
3.4. 关键点 26
3.5. 变式检验 27
3.6. 同类归类 27
3.7. Go 语言机制补充 27
3.8. 本次操作与当前状态 28
4. 为什么 spec 要写副作用，什么时候写 29
4.1. 费曼直觉 29
4.2. 硬核机制 29
4.2.1. 副作用有六种载体 29
4.2.2. 为什么必须写：四条推理链 29
4.2.3. 什么时候必须写：四问判据 30
4.2.4. 什么时候可以不写 30
4.2.5. Go 的真实样本 30
4.2.6. 失败语义是副作用的一部分 32
4.2.7. 映射回 `internal/db` 32
4.3. 陷阱与易错 32
4.4. 关键点 33
4.5. 变式检验 33
4.6. 同类归类 34
4.7. Go 语言机制补充 34
4.8. 本次操作与当前状态 34
5. Get spec 二轮评审 + go doc 的渲染语法 36
5.1. 费曼直觉 36
5.2. 硬核机制 36
5.2.1. 实测一：当前注释完全不被 go doc 显示 36
5.2.2. 实测二：去掉空行后，子项被合并 36
5.2.3. 实测三：`Preconditions:` 不是标题 37
5.2.4. 内容问题清单 37
5.2.5. 已经修好的两处 38
5.2.6. `go doc` 的渲染语法全表 38
5.2.7. 本机实测：gofmt 到底改什么 39
5.2.8. gofmt 做什么、不做什么 39
5.2.9. 建议的修改顺序（只给问题，不代写） 40
5.3. 陷阱与易错 40
5.4. 关键点 41
5.5. 变式检验 41

5.6. 同类归类 42
5.7. Go 语言机制补充 42
5.8. 本次操作与当前状态 42
6. 0101 spec 完成度盘点与下一步 44
6.1. 费曼直觉 44
6.2. 硬核机制 44
6.2.1. 完成度总表 44
6.2.2. 验证基线（刚跑过） 45
6.2.3. 三个已经做对的 45
6.2.4. 三个最大缺口 45
6.2.5. 依赖链：为什么下一步必须先定「空键 / 空值」 45
6.2.6. 两种选择的连锁后果 45
6.2.7. 下一步的完整顺序 46
6.2.8. 真实项目参照 46
6.3. 陷阱与易错 47
6.4. 关键点 47
6.5. 变式检验 47
6.6. 同类归类 48
6.7. Go 语言机制补充 48
6.8. 本次操作与当前状态 48
7. Set 的 spec 评审：三个矛盾与一个恒真布尔 50
7.1. 费曼直觉 50
7.2. 硬核机制 50
7.2.1. 先看渲染：注释又没显示 50
7.2.2. spec 承诺 vs 实测 50
7.2.3. 矛盾一：空切片键 51
7.2.4. 矛盾二：`Preconditions: nothing` 与零值 panic 51
7.2.5. 矛盾三：`updated` 是一个恒真布尔 51
7.2.6. 缺失的条款 52
7.2.7. 格式与实现细节 52
7.2.8. 真实项目对照 52
7.3. 陷阱与易错 53
7.4. 关键点 53
7.5. 变式检验 54
7.6. 同类归类 54
7.7. Go 语言机制补充 54
7.8. 本次操作与当前状态 55
8. Postcondition 与 effect 的区别 56
8.1. 费曼直觉 56
8.2. 硬核机制 56
8.2.1. 两栏各自回答什么 56
8.2.2. 包含关系：postcondition ⊇ effect 56

8.2.3. 四个样本拆解 57
8.2.4. 一个容易站错队的条款：别名 57
8.2.5. 为什么容易混 57
8.2.6. 在 Get / Set 的 spec 里怎么落地 57
8.2.7. 真实项目样本 58
8.3. 陷阱与易错 58
8.4. 关键点 59
8.5. 变式检验 59
8.6. 同类归类 59
8.7. Go 语言机制补充 60
8.8. 本次操作与当前状态 60
9. 忽略 Open / Close 后的 spec 全面检查（Get / Set / Del / AF / RI / Safety） 61
9.1. 费曼直觉 61
9.2. 硬核机制 61
9.2.1. 实测证据（通过 -overlay 注入，未写仓库） 61
9.2.2. 逐项问题清单 61
9.2.3. P0 展开 62
9.2.4. 已经做对的三处 63
9.2.5. 按 skill 的质量门槛对照 64
9.2.6. 三步修复顺序（忽略 Open / Close） 64
9.3. 陷阱与易错 64
9.4. 关键点 65
9.5. 变式检验 65
9.6. 同类归类 65
9.7. Go 语言机制补充 66
9.8. 本次操作与当前状态 66
10. 验收策略校准：当前测试强度下哪些要做、哪些推迟 68
10.1. 费曼直觉 68
10.2. 硬核机制 68
10.2.1. 当前测试到底约束了什么 68
10.2.2. 三类事项的重新分级 68
10.2.3. 「阶段末」必须有触发条件 69
10.2.4. 我之前超纲的要求（收回） 69
10.2.5. 修正后的下一步（只剩五件事） 70
10.2.6. 仍然建议现在做的一条：`Del` 的渲染 70
10.3. 陷阱与易错 70
10.4. 关键点 71
10.5. 变式检验 71
10.6. 同类归类 71
10.7. Go 语言机制补充 72
10.8. 本次操作与当前状态 72
Index of Listings 73

1. 两种 spec 写法有什么区别：JSDoc 的 `@param` 与 requires/effects 三段式

1.1. 费曼直觉

先给结论：**这不是两种规格语言的对决，而是同一份合同的两种排版方式。** 真正的区别只有两条：

1. 第二种把「调用者要先满足什么」（requires）和「函数保证什么」（effects / returns）拆成独立的栏；第一种把这些信息塞进了 `@param` 标签，没有单独的前置条件栏。
2. 第一种是 **就地修改入参** 的函数（mutator），第二种是 **返回新值、入参不变** 的函数（producer）。这一条才是实质差异，而且它决定调用方能不能继续使用旧值。

打个比方：第一种像贴在药盒上的便签——「吃了会犯困」写在药名旁边； 第二种像处方单——先写「病人不能空腹」，再写「服药后血糖下降」。 便签也能传达信息，但换一个药、换一个病人，栏位就不够用了。

> 📌 **核心要点** 判据不是「用没用 `@param` 」，而是三问：① 前置条件写了吗？② 会不会修改入参？③ 后置条件是可判真假的谓词，还是一串操作步骤？第一段代码的 `@param` 写法看起来随意，但它写的是**声明式谓词**，所以它并不比第二段差——它缺的是「前置条件」这一栏，而不是「规格性」。

1.2. 硬核机制

1.2.1. 先拆语义，再谈语法

把两段代码放进同一张四要素表，差别立刻显形：

要素	第一种：sort	第二种：toLowerCase
前置 requires	未写；隐含「array 是可变数组」	显式写 `nothing`
后置 effects	就地修改，修改后满足排序谓词	不修改入参，返回新数组 t
返回 returns	`void`	新数组 t，长度与 array 相同
变异	**是**（mutator）	**否**（producer）
失败 / 错误	未写（`array.sort()` 不会失败）	未写（映射不会失败）

这张表里只有「变异」一栏是真正的设计分叉。其余三栏的差别是**写法完整度**，不是语义分歧：第一种也可以补一行 `requires: array 非 nil`， 第二种也可以把 `@returns` 改写成 `effects:` 。

1.2.2. 为什么第一种把后置条件写进 `@param`

JSDoc 的 `@param` 本来只负责「这个参数代表什么」。材料里把「参数被就地修改后的状态」写进去，是为了在只有 JSDoc 一种载体的语言里表达后置条件。 代价有三个：

1. 后置条件被切成「每个参数一段」。如果函数修改三个参数，读者要拼三段才知道整体效果；

2. `@param` 的语气容易滑向**操作式**（「把它排序」），而不是**谓词**（「排序后满足……」）；
3. 工具不会校验这段文字。TypeScript 编译器只看类型，不看 `mutated into sorted order`。

第二种写法把 `effects` 放在函数级，正好避开这三点：函数整体做了什么、返回什么，一眼可见；`requires` 单独一栏，缺了就是明显的空白。

1.2.3. 关键纠偏：这两种都是声明式，不是「声明式 vs 操作式」

用户最容易踩的坑，是把「JSDoc 版」当成操作式、把「requires/effects 版」当成声明式。不是的。看它们各自的核心句子：

```
    array[i] <= array[j] for all 0 <= i < j < array.length

    t.length == array.length  and  t[i] == array[i].toLowerCase() for all i
```

Listing 1: 两种写法给出的都是可判真假的谓词

两句都是「对所有下标，某个等式 / 不等式成立」，都能直接翻译成测试断言。真正的操作式长这样：「取中间元素、递归排序左右两半、再合并」——它描述的是**怎么做**，换一种排序算法就不再成立。规格要的是「结果满足什么」，不是「用什么算法」。

类型	判据	例子
声明式	只描述结果满足的谓词，可判真假	`array[i] <= array[j]` 对所有 `i < j`
操作式	描述步骤或算法，换实现就失效	分治、先取中位数、再合并
欠定	谓词不够强，多个结果都满足	只说「排好序」，没说稳定性

第三种（欠定）不是错误：规格本来就允许欠定，只要调用方不依赖未承诺的细节。但 `sort` 这条规格恰好是欠定的——它没说相同元素的相对顺序。`sort.Slice` 的官方注释就明写了「不保证稳定」，而 `slices.Sort` 没有写。这不是语法差异，是契约强弱的差异。

1.2.4. 互译：同一份契约的两种载体

把第一种翻译成三段式，只需把 `@param` 里那句话提到 `effects`，再补 `requires`：

```
    sort(array: Array<string>): void
    requires:
      array is a mutable array of strings
    effects:
      mutates array so that array[i] <= array[j] for all 0 <= i < j <
array.length
    returns:
      nothing
```

Listing 2: 第一种写法的三段式版本：语义不变，只是把后置条件提到函数级

反过来，把第二种翻成 JSDoc，关键是**在注释里明说不修改入参**，并用类型把这条承诺加固：

```
  /*
   * Returns a new array without modifying the input.
   * @param array - read-only; the function does not modify it.
   * @returns a new array t with t.length == array.length and
   *     t[i] == array[i].toLowerCase() for all i.
   */
  function toLowerCase(array: ReadonlyArray<string>): Array<string> {
    return array.map(s => s.toLowerCase());
  }
```

Listing 3: 第二种写法的 JSDoc 版本：ReadonlyArray 让「不修改」从注释变成类型检查

注意最后的 `ReadonlyArray`：这是两种写法少有的实质能力差。JSDoc 的文字承诺无法被编译器检查，而类型可以把「不修改」变成编译期约束。Go 里没有 `const` 参数，所以这条只能靠注释 + 拷贝来兑现——下文有样本。

1.2.5. Go 里没有这两种标签，合同写在哪

Go 既没有 `@param`，也没有 `requires:` 关键字。规格只能写在紧邻声明的 `//` 注释里，`go doc` 负责把它排版出来，但不做任何语义校验。所以「写哪种」在 Go 里退化成「注释里有没有把前置、后置、变异、错误四件事说清」。

真实样本一：producer，明说返回副本。

```
    // ToLower returns a copy of the byte slice s with all Unicode letters
mapped to
    // their lower case.
    func ToLower(s []byte) []byte
```

Listing 4: 标准库 bytes.ToLower：一句话同时给了后置条件（映射小写）和别名条款（copy）

`returns a copy` 这五个字就是 `effects`：调用方之后修改 `s` 不会影响返回值，修改返回值也不会影响 `s`。第二种写法里的 `returns a new array t` 与它同构。

真实样本二：mutator，但注释没写明就地修改。

```
    // Sort sorts a slice of any ordered type in ascending order.
    // When sorting floating-point numbers, NaNs are ordered before other
values.
    func Sort[S ~[]E, E cmp.Ordered](x S)
```

Listing 5: 标准库 slices.Sort：后置条件写了，但「就地修改 x」只能从实现看出

这正好是第一种写法的 Go 版本，而且暴露了它的缺陷：`Sort` 修改了 `x`，注释却没有一句 `modifies x in place`。读者必须知道 `slices.Sort` 的惯例，或者去读源码，才能确定自己手里的切片会不会变。这就是为什么「变异」这一栏必须在合同里显式写出来。

真实样本三：把 nil 语义写死。

```
// Clone returns a copy of the slice.
// The elements are copied using assignment, so this is a shallow clone.
// The result may have additional unused capacity.
// The result preserves the nilness of s.
func Clone[S ~[]E, E any](s S) S
```

Listing 6: 标准库 slices.Clone：连 nil 进 nil 出都写进了合同

`The result preserves the nilness of s` 是一句典型的声明式后置条件：它没有说「怎么拷贝」，只说了结果的性质。这条正好对应 `internal/db` 里 `Get` 未命中时该返回 `nil` 还是 `[]byte{}` 的决策。

1.2.6. 映射回 `internal/db`：先分类，再填栏

0101 的五个方法按「变异」分类如下。分类决定注释里必须有哪一栏，但每栏的内容仍然要你自己定（这是作业，不是抄写）：

方法	类型	合同里不能缺的栏
`Get`	producer	未命中时 `val` 是 nil 还是空切片；`ok` 与 `err` 的分工；返回值是否别名 rep
`Set`	mutator（改 rep，不改入参）	`updated` 的语义；`key` / `val` 的拷贝政策；空键 / 空值是否合法
`Del`	mutator	`deleted` 表示「键存在」还是「删除成功」；重复删除的结果
`Open`	mutator（重建 rep）	零值能否直接用；重复调用是否清空；error 何时非 nil
`Close`	生命周期操作	关闭后能否继续用；是否幂等；error 何时非 nil

注意 `Set` 的入参政策：它修改的是**内部 map**，不修改调用方传进来的切片。 这是 producer / mutator 之外的第三个维度——「rep 变了，但入参没变」。 合同里要分别写清楚，否则客户会误以为 `Set` 之后可以复用或释放 `val`。

1.3. 陷阱与易错

⚠ **陷阱预警 别把载体当语义。** JSDoc 与 requires/effects 只是排版。 把第一种换成三段式，函数依然修改入参；把第二种换成 JSDoc，函数依然返回新数组。 评审时先问「前置、后置、变异、错误写了吗」，而不是「格式对不对」。

⚠ **陷阱预警「就地修改」必须显式写。** `slices.Sort` 的注释只说了结果有序，没说 `x` 被改。客户如果同时持有另一个指向同一底层数组的切片，会看到意料之外的变化。Go 里没有 `ReadonlyArray`，这条只能靠注释和拷贝来保证。

⚠️ **陷阱预警 `@param` 写副作用是权宜之计。** 它把后置条件切碎，而且工具不校验。在 TypeScript 里可以用 `ReadonlyArray` / `readonly` 把一部分承诺变成类型检查；在 Go 里没有对应机制，只能写进注释并用测试锁住。

⚠️ **陷阱预警 声明式不等于短。**「返回新数组」是声明式，「先 new 再遍历再 push」也是描述，后者是操作式。判据是可判真假的谓词，不是句子长短。

1.4. 关键点

📌 **核心要点** 回答原问题：两种写法的区别是「分区方式」+「变异语义」。第一种按参数分区，把后置条件写进 `@param`，且函数就地修改入参；第二种按前置 / 后置 / 返回分区，函数不改入参。两者都可以是声明式；真正决定合同质量的是四栏是否齐全，而不是用哪套标签。

维度	JSDoc 版（第一种）	REQUIRES/EFFECTS 版（第二种）
分区	按参数	按前置 / 后置 / 返回
前置条件	没有独立栏，常省略	独立 `requires` 栏
变异	写在 `@param` 描述里	写在 `effects` 里
机器检查	类型可查，文字不校验	不校验（课程规格语言）
Go 对应	doc comment 自由段落	同左，用 `Precondition:` / `Returns` 分段

1.5. 变式检验

变式 1：把 `sort` 改成返回新数组，规格要改哪几行？ `effects` 从「mutates array」改成「returns a new array t」，`returns` 从 `nothing` 改成 `t`，`requires` 可以保留；调用方代码也要改，因为原来的返回值是 `void`。这就是「变异条款改一行，调用方全要动」。

变式 2：`toLowerCase` 改成就地修改，Go 里怎么写？ 函数签名仍是 `func ToLowerInPlace(s []byte)`，合同要写 `modifies s in place`，并且要说明「如果 s 与别的切片共享底层数组，那些切片也会变」。这类函数在 Go 里通常直接用 `bytes.ToLower` 的返回新值版本，因为别名分析更难。

变式 3：`sort` 的规格欠定在哪？ 它没写相同元素的相对顺序，所以稳定排序和不稳定排序都满足。要补强就加一句「equal elements keep their relative order」，但代价是实现只能用稳定排序，性能可能下降——这是 C06「规格强弱」的典型取舍。

变式 4：能不能只写后置条件，不写前置？ 可以，但要确认后置条件对所有输入都成立。`sort` 对 nil 切片也成立（排序 nil 无事发生），所以 `requires` 可以写 `nothing`；如果函数要求「数组非空」，就必须写前置，否则调用者传空数组时行为未定义。

变式 5：`internal/db` 的 `Get` 用哪种风格写？ Go 只有一种载体（doc comment），所以问题不是选风格，而是选条款：`val` 未命中返回 nil 还是空切片、`ok` 与 `err` 分工、返回值是否别名 rep。 先答这三问，再决定注释怎么写。

1.6. 同类归类

体系	规格载体	能否被工具检查
JSDoc	`@param` / `@returns` 标签	类型可查；文字不校验
6.031 / 6.102	requires / effects / returns 三段式	不校验（人工评审）
Go	紧邻声明的 `//` 注释	`go doc` 只排版，不校验语义
Eiffel DbC	`require` / `ensure` 关键字	可运行时检查（契约编译选项）
Rust	doc comment + `# Panics` / `# Errors`	`cargo test --doc` 可执行示例

真实项目的选择也不统一：Go 标准库用自由段落（`bytes.ToLower`、`slices.Clone`），Eiffel 把契约做成语言关键字，Rust 用文档段落 + doctest。 差异在于**工具能替你检查多少**；不能检查的部分，只能靠测试和评审。

1.7. Go 语言机制补充

Go 没有 `const` 参数，也没有 `readonly` 切片。 函数收到 `[]byte` 时， 得到的是一个切片头（指针、长度、容量）的副本；底层数组仍然共享。 所以「不修改入参」只能靠约定 + 拷贝：`bytes.Clone` / `slices.Clone` / `append([]byte(nil), s...)`。 这是 `internal/db` 里 `Set`、`Get` 要 `bytes.Clone` 的根本原因。

`string([]byte)` 会复制字节。字符串不可变，转换时复制，所以 `mem[string(key)]` 之后调用方修改 `key` 不影响 map 的键。 这条理由要写进 Safety，否则读者会问「为什么 `val` 拷贝了， `key` 没拷贝」。

map 是引用类型。 `KV` 的 `mem` 字段赋值给另一个变量不会复制底层哈希表； `Set` 修改 map 后，所有持有同一个 map 的地方都能看到。 所以 `Set` 是 mutator，但它的「变异」作用在 rep 上，不是入参上。

`slices.Clone` 会保留 nil。 官方注释写 `The result preserves the nilness of s`， 这意味着 `Clone(nil)` 返回 `nil` 而不是空切片。 `Get` 未命中时 `val == nil` 正是 `bytes.Clone(nil)` 的结果；如果合同要求空切片，就要显式写成 `[]byte{}`。

`go doc` 只排版，不验证。 注释里的 `requires` 写错了， `go doc` 照样显示， `go vet` 也不报。 所以规格正确性只能由测试和评审保证——这也是为什么 0101 的 P0 里「先定条款、再写测试」。

6.031 对照。 本题对应 R06 规格（前置 / 后置条件、异常与 null 属于规格）、 R05/R07 设计规格（声明式 / 操作式、强 / 弱、确定性 / 欠定）。R06 的形式正是 `requires` / `effects` / `returns`；6.102 的 R04/R05 用 JSDoc 承载同一组概念。两者不是替代关系：一个强调「合同有哪些栏」，一个强调「在 TS 里怎么落地」。

1.8. 本次操作与当前状态

已做的文件操作：

1. 加载 `software-construction` skill，读取 `references/concept-atlas.md` 的 C05 规格、C06 设计规格两节；
2. 加载 `golang-documentation` skill，核对 doc comment 的段落形状与 `TODO` 惯例；
3. 用 `go doc -src` 读取 `bytes.ToLower`、`slices.Sort`、`slices.Clone`、`strings.Clone`、`sort.Slice` 的原文注释；
4. 新建本文件 `note/2026-11-12.typ`（当日首个问题），沿用 2026-11-10 的完整头部；
5. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `go doc -src bytes.ToLower` → `ToLower returns a copy of the byte slice s ...`；
2. `go doc -src slices.Sort` → 只写排序结果与 NaN 顺序，未写「就地修改」；
3. `go doc -src slices.Clone` → `The result preserves the nilness of s`；
4. `go doc -src sort.Slice` → 写了「不保证稳定」，同样未写就地修改；
5. 本机 `go.etcd.io/bbolt` 不在模块缓存中，因此未引用 bbolt 的 `Put` 契约样本；
6. `XDG_CACHE_HOME=/home/zz/.cache typst compile 2026-11-12.typ` → 待编译确认（见下）。

需要你自己动手的：

1. 先回答 0101 的决策 D：`Get` 未命中时 `val` 返回 nil 还是空切片？`ok` 与 `err` 如何分工？
2. 再回答决策 A / B / C（`updated` 语义、空键 / 空值、生命周期）；
3. 填注释时，按「前置 / 后置 / 变异 / 错误」四栏自检，而不是按 JSDoc 或三段式的格式自检；
4. 写完后用 `go doc ./internal/db KV.Get` 检查摘要与条款是否通顺。

2. Go 的 spec 怎么写：标准库与 Google 开源项目的真实样本

2.1. 费曼直觉

Go 没有 `requires:` / `effects:` 关键字，也没有 `@param` 标签。 合同的唯一载体是**紧邻声明的 `//` 注释**，`go doc` 只负责排版，不校验语义。 所以问题变成：Google 自己的代码，用自由段落把四栏（前置、后置、变异、错误）写在哪。

先给结论，Google 的做法可以归纳成一句话：

> 📌 **核心要点 只写「不明显且会害人」的部分。** 参数不必逐个解释；但「能不能并发用」「谁负责关」「返回什么错误类型」「会不会改入参」 这四件事必须写——因为客户无法从函数名和类型推出来。

这与上一节的三段式并不冲突：三段式告诉你「有哪些栏」，Go 的惯例告诉你「每栏该写多深」。Google Style Guide 甚至明确说，参数逐条解释往往是噪音。

2.2. 硬核机制

2.2.1. 规则先行：Google Go Style Guide 的四条硬要求

出处：`https://google.github.io/styleguide/go/best-practices#documentation` （本机抓取的原文，下面引号内的句子改为中文引号，其余照录）。

规则	原文要点	对应哪一栏
参数	不必逐个列出；只解释容易错或不明显的字段与参数，说明「为什么值得注意」	前置条件
并发	「Whether an API is safe for use by multiple goroutines is part of its contract.」	使用约束
清理	「Document any explicit cleanup requirements that the API has.」否则会资源泄漏	生命周期
错误	要写清返回的错误类型，以及是值还是指针接收者，客户才能用 `errors.Is` / `errors.As`	错误语义

这四条正好覆盖了 Go 里最常缺的四类信息。注意它没有要求「写全每个参数」—— 这与 6.031 的 `requires` 栏精神一致：前置条件要写，但只写调用者必须知道的那部分。

2.2.2. 样本一：接口级合同，`io.Reader`

`io.Reader` 的注释是教科书级样本：它规定的不只是一个方法，而是**所有实现者与调用者的共同协议**。

```go
// Read reads up to len(p) bytes into p. It returns the number of bytes
// read (0 <= n <= len(p)) and any error encountered.
//
// When Read encounters an error or end-of-file condition after
// successfully reading n > 0 bytes, it returns the number of
// bytes read. It may return the (non-nil) error from the same call
// or return the error (and n == 0) from a subsequent call.
//
// Callers should always process the n > 0 bytes returned before
// considering the error err.
//
// Implementations must not retain p.
type Reader interface {
  Read(p []byte) (n int, err error)
}
```

Listing 7: GOROOT `src/io/io.go` 的 Reader 注释（节选）

四栏在这里如此分布：

1. **后置条件**：`0 <= n <= len(p)`；错误可以本次返回，也可以下次返回；
2. **调用者义务**：先处理 `n > 0` 的字节，再看 `err`；
3. **实现者义务**：不得保留 `p`（否则调用方复用缓冲区会出问题）；
4. **边界**：`len(p) == 0` 时应返回 `n == 0`，可以带已知错误。

`Implementations must not retain p` 是典型的**别名条款**——它约束的是实现者，而不是调用者。三段式写法里没有专门的「实现者义务」栏，Go 用一句 `must not` 补上。

2.2.3. 样本二：使用约束，`sync.Mutex`

```go
// A Mutex is a mutual exclusion lock.
// The zero value for a Mutex is an unlocked mutex.
//
// A Mutex must not be copied after first use.
```

Listing 8: GOROOT `src/sync/mutex.go`：零值可用 + 不得复制

两句话，两个合同：

1. **零值语义**：`var mu sync.Mutex` 就能用，不需要构造函数；
2. **使用约束**：复制会破坏互斥，所以禁止复制。

第二条不是「建议」，是可被工具检查的硬约束。`sync` 在结构体里放了 `_ noCopy` 哨兵，`go vet` 的 copylocks 分析器据此报警。本机实测：

```
    type S struct{ mu sync.Mutex }

    func main() {
      var a S
      b := a
      _ = b
    }
```

Listing 9: `$TMPDIR` 下的最小复现：复制含 Mutex 的结构体

```
    main.go:9:7: assignment copies lock value to b: example.com/mutexcopy.S
contains sync.Mutex
    main.go:10:6: assignment copies lock value to _: example.com/mutexcopy.S
contains sync.Mutex
```

Listing 10: `go vet ./...` 的真实输出（退出码 1）

这是 Go 特有的「把合同变成静态检查」的路线：注释写约束，类型加哨兵，工具来执行。`internal/db` 的 `KV` 含 map，复制 `KV` 会让两个对象共享同一张表；如果以后要防这类错误，可以照抄 `noCopy` 的做法。

2.2.4. 样本三：同名不同合同，`os.File.Close` 与 `sql.Rows.Close`

两个 `Close`，合同完全相反：

```
    // Close closes the [File], rendering it unusable for I/O.
    // On files that support [File.SetDeadline], any pending I/O operations
will
    // be canceled and return immediately with an [ErrClosed] error.
    // Close will return an error if it has already been called.
```

Listing 11: GOROOT `src/os/file.go`：关闭后不可用，重复调用返回错误

```
    // Close closes the [Rows], preventing further enumeration. If
[Rows.Next] is called
    // and returns false and there are no further result sets,
    // the [Rows] are closed automatically and it will suffice to check the
    // result of [Rows.Err]. Close is idempotent and does not affect the
result of [Rows.Err].
```

Listing 12: GOROOT `src/database/sql/sql.go`：幂等，且不影响 Err 的结果

四个差异点，全都要在合同里写明：

维度	os.File.Close	sql.Rows.Close
关闭后可用性	不可用（rendering it unusable）	不可继续枚举
重复调用	返回错误	幂等（idempotent）
自动关闭	无	`Next` 返回 false 后自动关闭
对错误状态的影响	无额外承诺	不影响 `Rows.Err` 的结果

这就是为什么「Close 后还能不能用」不能靠猜。两个都叫 `Close`，一个幂等、一个不幂等；一个影响后续 I/O，一个不影响错误查询。`internal/db` 的 `Close` 当前是 no-op，合同必须写清它属于哪一种，而不是留一句「关闭存储」。

2.2.5. 样本四：并发安全，`net/http.Client`

```
1    // A Client is an HTTP client. Its zero value ([DefaultClient]) is a
2    // usable client that uses [DefaultTransport].
3    //
4    // The [Client.Transport] typically has internal state (cached TCP
5    // connections), so Clients should be reused instead of created as
6    // needed. Clients are safe for concurrent use by multiple goroutines.
```

Listing 13: GOROOT `src/net/http/client.go`（节选）：零值可用 + 可复用 + 并发安全

这三句合起来是一个完整的「对象生命周期合同」：怎么创建（零值可用）、怎么复用（不要每次新建）、怎么共享（多 goroutine 安全）。

注意 Style Guide 的补充规则：**只读操作默认并发安全，变异操作默认不安全**，所以只在「不明确」或「API 自己提供同步」时才需要写。`Client` 属于后者（内部缓存连接），所以必须写；`bytes.Buffer.Len` 属于前者，可以不写。

2.2.6. 样本五：前置条件的两条路线，`proto.Unmarshal` 与 `json.Unmarshal`

同样是「把数据解到目标对象」，Google 的两个库选了不同路线。

```
1    // Unmarshal parses the wire-format message in b and places the result
in m.
2    // The provided message must be mutable (e.g., a non-nil pointer to a
message).
3    //
4    // See the [UnmarshalOptions] type if you need more control.
5    func Unmarshal(b []byte, m Message) error
```

Listing 14: `google.golang.org/protobuf@v1.36.12/proto/decode.go`：写成前置条件

```
1    // Unmarshal parses the JSON-encoded data and stores the result
2    // in the value pointed to by v. If v is nil or not a pointer,
3    // Unmarshal returns an [InvalidUnmarshalError].
```

Listing 15: GOROOT `src/encoding/json/decode.go`：写成返回错误

路线	写法	违约后果
前置条件	`must be mutable`	行为未定义；调用者自己负责
返回错误	`returns an InvalidUnmarshalError`	实现检查，客户可以处理

两条都合法，选择取决于「实现愿不愿意为违约付出检查成本」。`internal/db` 的 `Set` 收到 nil map 时会 panic，属于「没写前置条件、也没返回错误」的第三种——这是最差的组合，合同里必须补上其中一条。

2.2.7. 样本六：明确写出「不承诺什么」，`cmp.Diff`

规格不只说「保证什么」，也要说「不保证什么」。Google 的 `go-cmp` 写得很直白：

```
    // Diff returns a human-readable report of the differences between two
values:
    // y - x. It returns an empty string if and only if Equal returns true
for the
    // same input values and options.
    //
    // The output is displayed as a literal in pseudo-Go syntax.
    //
    // Do not depend on this output being stable. If you need the ability to
    // programmatically interpret the difference, consider using a custom
Reporter.
```

Listing 16: `github.com/google/go-cmp@v0.7.0/cmp/compare.go`（节选：中间关于每行前缀的说明已略）

`returns an empty string if and only if Equal returns true` 是强后置条件；`Do not depend on this output being stable` 是显式的欠定声明；最后一句还给了替代方案（custom Reporter）。

这三件套值得照抄：**承诺、拒绝承诺、给出替代路径。** `internal/db` 的 `Get` 也该写「返回值是副本，但在你修改它之前有效」这类边界，而不是只说「返回对应的值」。

2.2.8. 样本七：错误类型必须写，`os.Chdir`

```
    // Chdir changes the current working directory to the named directory.
    // If there is an error, it will be of type [*PathError].
```

Listing 17: GOROOT `src/os/file.go`：错误类型属于合同

为什么这行重要？因为客户要写 `errors.As(err, &pathErr)`。如果注释不说「是 `*PathError`」，客户只能猜，猜错就永远匹配不到。Style Guide 明确说：返回类型写 `error` 而不是 `*PathError`，是 nil interface 的惯例；但注释里必须点明具体类型。

`internal/db` 现在三个方法都返回 `error` 且恒 nil。合同要么写「当前不产生错误」，要么定义哨兵错误并说明何时返回——两者都比空着强。

2.2.9. 样本八：反例，变异没写进合同

```
    // Sort sorts a slice of any ordered type in ascending order.
    // When sorting floating-point numbers, NaNs are ordered before other
values.
    func Sort[S ~[]E, E cmp.Ordered](x S)
```

Listing 18: GOROOT `src/slices/sort.go`：写了结果，没写就地修改

对比同一个包里的 `Clone`：

```
// Clone returns a copy of the slice.
// The result preserves the nilness of s.
func Clone[S ~[]E, E any](s S) S
```

Listing 19: GOROOT `src/slices/slices.go`（节选）：明说返回副本，连 nil 语义都写死

`Sort` 修改 `x`，注释没有任何一句 `modifies x in place`；而 `Clone` 用 `returns a copy` 把别名关系说死。这组对照说明：**变异条款的缺失在标准库里也存在**——所以不能拿「标准库没写」当自己不写的理由，反而应该把它当反例。

2.2.10. 汇总：八个样本覆盖哪一栏

样本	对应栏	可照抄的句式
`io.Reader`	接口级全部	`Implementations must not retain p`
`sync.Mutex`	零值 + 使用约束	`The zero value is ...` / `must not be copied`
`os.File.Close`	生命周期	`rendering it unusable for I/O`
`sql.Rows.Close`	幂等	`Close is idempotent and does not affect ...`
`http.Client`	并发 + 复用	`safe for concurrent use by multiple goroutines`
`proto.Unmarshal`	前置条件	`The provided message must be mutable`
`json.Unmarshal`	错误返回	`returns an InvalidUnmarshalError`
`cmp.Diff`	不承诺 + 替代	`Do not depend on this output being stable`
`os.Chdir`	错误类型	`it will be of type *PathError`
`slices.Sort`	变异缺失的反例	（反面教材，不要照抄）

2.2.11. 映射回 `internal/db`：五个方法该参照谁

方法	参照样本		当前缺的句子
`Get`	`slices.Clone` `io.Reader`	+	未命中时 `val` 的 nil 语义；`ok` 与 `err` 分工；返回值是副本
`Set`	`proto.Unmarshal` `slices.Sort`	+	入参是否拷贝；`updated` 语义；空键 / 空值的后果
`Del`	`sql.Rows.Close`	（幂等）	`deleted` 表示存在还是成功；重复删除的结果
`Open`	`sync.Mutex`	（零值）	零值能否直接用；重复调用是否清空；error 何时非 nil
`Close`	`os.File.Close` `sql.Rows.Close`	vs	关闭后能否继续用；是否幂等；error 何时非 nil

注意 `Set` 的参照里同时有 `proto.Unmarshal`（前置条件）和 `slices.Sort`（变异未写明）：前者是正面样本，后者是反例。`Set` 既改 rep 又接收 `[]byte` 入参，两个维度都要交代，不能只写一句「存储键值」。

2.3. 陷阱与易错

⚠️ **陷阱预警「Go 的注释不是规格」是最大的误解。** Go 没有独立的规格文件，doc comment 就是客户能读到的唯一合同；`go doc` 渲染它，测试和 `go vet` 都不会替你检查它。 不写注释 = 没有合同，而不是「合同在类型里」。

⚠️ **陷阱预警 零值可用不是默认。** `sync.Mutex` 、 `bytes.Buffer` 、 `http.Client` 的零值可用，但 `KV{}` 直接 `Set` 会 panic（nil map），`os.File{}` 也不能用。 合同必须逐个说明，不能从「Go 的类型设计」推断。

⚠️ **陷阱预警 同名方法可以有相反合同。** 两个 `Close` ： 一个重复调用报错，一个幂等。 `Close` 这个名字不携带任何语义，必须靠注释区分。

⚠️ **陷阱预警 只写保证、不写拒绝承诺。** `cmp.Diff` 用 `Do not depend on ...` 明确划界。省略这句，客户就会把格式化输出当稳定 API 来解析，下一次升级就炸。

⚠️ **陷阱预警 变异条款最容易漏。** `slices.Sort` 是标准库里的现成反例。 凡是接收 `[]byte` 、 `[]T` 、map、指针的函数，都要问一句「我改没改它」。

2.4. 关键点

📌 **核心要点** Go 的 spec = 紧邻声明的 doc comment + 四栏内容（前置 / 后置 / 变异 / 错误）+ 三条补充 （并发安全、清理责任、拒绝承诺）。 Google 的实践是「只写不明显且会害人的部分」，而不是逐参数罗列。

栏位	Go 里怎么写	样本
前置条件	`must be ...` / `If v is nil, returns ...`	`proto.Unmarshal` 、 `json.Unmarshal`
后置条件	陈述句 + 可判真假的谓词	`io.Reader` 、 `cmp.Diff`
变异 / 别名	`returns a copy` / `must not retain p`	`bytes.ToLower` 、 `io.Reader`
生命周期	零值、复用、关闭后行为、幂等	`Mutex` 、 `File.Close` 、 `Rows.Close`
并发	`safe for concurrent use`	`http.Client`
错误	具体类型 + 何时返回	`os.Chdir`

2.5. 变式检验

变式 1：`Get` 未命中返回 nil，注释该怎么写才能防止客户 append？ 写清 `val` 是副本、未命中时为 nil；客户若 `append` 会得到新切片，不影响 rep。 再补一句 `ok` 是唯一可靠的命中判据——`val` 为 nil 时也可能是命中且值为空。

变式 2：`Set` 要不要写「不得修改入参」？ 要，但方向相反：`Set` 承诺**自己会拷贝**，所以客户在 `Set` 之后修改 `val` 不影响 store。这是实现给客户的保证，不是客户义务。写成 `stores a copy of val` 即可。

变式 3：`Close` 该照抄 `os.File.Close` 还是 `sql.Rows.Close`？ 取决于你要不要「关闭后失效」。现在实现是 no-op，最省事的合同是「Close 是幂等的， 关闭后对象仍可用」——但这是一条真实承诺，测试要能验证；若不想承诺，就加 `closed` 状态。

变式 4：`Open` 重复调用会清空数据，要不要写进合同？ 必须写，因为客户可能 `defer kv.Open()` 或重复初始化。 合同要么说「重复调用会清空」，要么改实现为「已初始化则 no-op」；不能沉默。

变式 5：为什么 Google 说参数不必逐个解释？ 因为类型和名字已经表达了显然的信息，重复一遍是噪音；真正要写的是「为什么这个参数值得注意」——范围、别名、并发、生命周期。 这与三段式的 `requires` 不矛盾：requires 只写调用者必须知道的部分。

2.6. 同类归类

合同类型	代表样本	典型句式
接口协议	`io.Reader`	实现者 `must` / 调用者 `should`
使用约束	`sync.Mutex`	`must not be copied after first use`
生命周期	`os.File.Close` 、 `http.Client`	`rendering it unusable` / `reused instead of created`
幂等	`sql.Rows.Close`	`is idempotent`
前置条件	`proto.Unmarshal`	`must be mutable`
错误语义	`json.Unmarshal` 、 `os.Chdir`	`returns an XError` / `will be of type *PathError`
欠定声明	`cmp.Diff`	`Do not depend on ... being stable`

同一件事在不同项目里有不同路线，这也是要写清合同的原因：`proto.Unmarshal` 用前置条件， `json.Unmarshal` 用返回错误； `os.File.Close` 不幂等， `sql.Rows.Close` 幂等。 客户无法从名字推断，只能读注释。

2.7. Go 语言机制补充

doc comment 的关联规则。 注释必须紧邻声明，中间不能有空行或别的注释块；第一行是摘要， `go doc` 用它做索引。 `go doc -all <包>` 显示全部导出符号的注释。 本机可用 `go doc -src <符号>` 直接看源码与注释，是核对原文最快的方式。

`noCopy` 与 `go vet`。 `sync.Mutex` 结构体里有 `_ noCopy` 哨兵，`go vet` 的 copylocks 分析器据此报告复制行为。这是 Go 把「使用约束」变成静态检查的少数例子。自定义类型若也有「不得复制」的约束，可以照抄这个模式。

零值语义。 Go 的类型零值总是存在，但「零值可用」是设计选择，不是语言保证。`sync.Mutex` 零值可用、`bytes.Buffer` 零值可用、`http.Client` 零值可用；`KV` 零值的 `mem` 是 nil，`Set` 会 panic。合同必须逐个说明。

`error` 与 `errors.Is` / `errors.As`。 返回 `error` 接口时，注释要写具体类型，客户才能用 `errors.As` 取细节。哨兵错误用 `errors.Is` 比较。`internal/db` 当前的注释掉的 `ErrNotFound` / `ErrUpdatedFailed` 就是哨兵错误的雏形。

Example 测试是可执行文档。 `func ExampleXxx()` 写在 `_test.go` 里，`go test` 会运行它并比对 `// Output:`。`internal/db` 现在没有 Example，阶段末可以考虑给 `Get` / `Set` 各加一个——它同时是文档和测试。

6.031 对照。 本节样本对应 R06 规格（前置 / 后置 / 异常）与 R07 设计规格（强 / 弱、确定性 / 欠定）。`cmp.Diff` 的「不承诺稳定」正是 R07 的欠定声明；`os.File.Close` 与 `sql.Rows.Close` 的对照是 R06「同一操作可以有不同合同」的实例。

2.8. 本次操作与当前状态

已做的文件操作：

1. 从 GOROOT 读取 `io.Reader`、`sync.Mutex`、`os.File.Close`、`database/sql.Rows.Close`、`net/http.Client`、`time.Timer.Reset`、`encoding/json.Unmarshal`、`os.Chdir`、`slices.Sort`、`slices.Clone`、`bytes.ToLower` 的原文注释；
2. 从模块缓存读取 `google.golang.org/protobuf@v1.36.12/proto/decode.go` 与 `github.com/google/go-cmp@v0.7.0/cmp/compare.go` 的原文注释；
3. 抓取并阅读 Google Go Style Guide 的 Documentation 章节（`https://google.github.io/styleguide/go/best-practices#documentation`）；
4. 在 `$TMPDIR/mutexcopy` 建立最小复现，验证复制含 `sync.Mutex` 的结构体会被 `go vet` 报告；
5. 追加本一级标题到 `note/2026-11-12.typ`；
6. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `go doc -src io.Reader` → 确认 `Implementations must not retain p` 等条款；
2. `go doc -src sync.Mutex` → 确认零值可用与 `must not be copied`，结构体含 `_ noCopy`；
3. `go doc -src os.File.Close` → 确认「不可用 + 重复调用返回错误」；
4. `go doc -src database/sql.Rows.Close` → 确认 `Close is idempotent`；
5. `go doc -src net/http.Client` → 确认 `safe for concurrent use by multiple goroutines`；
6. `go doc -src encoding/json.Unmarshal` → 确认非法 `v` 返回 `InvalidUnmarshalError`；
7. `go doc -src slices.Sort` → 确认注释未写就地修改；
8. `grep` 模块缓存 → `proto.Unmarshal` 的 `must be mutable`、`cmp.Diff` 的 `Do not depend on this output being stable`；

9. `cd $TMPDIR/mutexcopy && go vet ./...` → 退出码 `1`，输出 `assignment copies lock value to b: example.com/mutexcopy.S contains sync.Mutex`；
10. 本机模块缓存有 `google.golang.org/protobuf@v1.36.12`、`github.com/google/go-cmp@v0.7.0`、`github.com/google/uuid@v1.6.0`、`github.com/google/pprof`，未安装 `grpc-go`。

需要你自己动手的：

1. 从上面的「映射回 internal/db」表里挑 `Get`，按 `slices.Clone` + `io.Reader` 的句式写三句条款；
2. 写完后用 `go doc ./internal/db KV.Get` 检查：摘要是否一句话、条款是否覆盖未命中与别名；
3. 再决定 `Close` 照抄哪一条路线（幂等且可用，或关闭后失效），并让实现与注释一致；
4. 阶段末可以考虑给 `Get` 加一个 `ExampleKV_Get`，同时当文档和测试。

3. Get 的 spec 首轮评审：一条假前置条件与两处缺条款

3.1. 费曼直觉

先给判定：**结构对了，但合同里有三处会误导客户。**

你已经做到三件对的事：把未命中单独成段、写了 `ok` 的布尔含义、明确未命中时 `err == nil`。但这份 spec 现在像一张「写着禁止停车、却没人执法」的告示：它声称调用者必须传非空 key，实现却默默接受空 key；它说 `val == nil` 表示未命中，但存一个 nil 值之后这个判据就失效；它没说出返回值是副本还是内部切片。

> 📌 **核心要点** 规格的价值不在「句子多」，而在「每句都能判决真假、且与实现一致」。这份 spec 的三条 P0 都是同一个毛病：**写了客户会依赖、实现却不保证的话**。

3.2. 硬核机制

3.2.1. 逐句核对

原文	判定	问题
`Get returns the new value associated with key.`	P1	`new` 含糊；摘要没写未命中，`go doc` 索引只显示这一行
`It requires that the length of key is more than or equal 1.`	**P0**	前置条件未被执行；空 key 实际可查
`ok is true represents the key is present, err is nil.`	P1	语法别扭；`err is nil` 应独立成句
`val == nil, ok = false, err == nil.`	**P0**	命中且值为 nil 时也是 `val == nil`，无法区分
`Other condition, ... the err will descripe the type of error`	P1	实现没有任何 error 分支，这是不可达承诺
（未写）	**P0**	返回值是否别名 rep 完全没提

3.2.2. 实测：四个探针

为了不靠猜，我用 `-overlay` 把一份探针测试注入 `internal/db` 跑了一遍，仓库里不落任何文件：

```
var zero KV
val, ok, err := zero.Get(nil)                  // 零值对象，空 key
println(val == nil, ok, err == nil)                // 依次打印三个布尔值

var kv KV
kv.Open()
_, ok, err = kv.Get(nil)                           // 已初始化，空 key
println(ok, err == nil)

kv.Set([]byte{'k'}, nil)                           // 存一个 nil 值
val, ok, err = kv.Get([]byte{'k'})
println(val == nil, ok, err == nil)

orig := []byte{'v', '1'}
kv.Set([]byte{'a'}, orig)
orig[0] = 'X'                                      // 改入参
stored, _, _ := kv.Get([]byte{'a'})
println(string(stored))
```

Listing 20: 探针测试的核心片段（通过 `-overlay` 注入，未写入仓库）

场景	实际返回	说明
零值 `KV{}` 上 `Get`	`nil, false, nil`	不 panic；nil map 读合法
空 key `Get`	`nil, false, nil`	不报错；前置条件等于不存在
存了 nil 值后 `Get`	`nil, true, nil`	`val == nil` 但 `ok == true`
改入参后再 `Get`	仍是 `v1`	入向拷贝成立
改返回值后再 `Get`	仍是 `v1`	出向拷贝成立

3.2.3. P0 一：前置条件没有被执行

spec 说 `len(key) >= 1`，但实现第 69 行只有一句 `kv.mem[string(key)]`，没有长度检查，也没有返回错误。探针证明空 key 返回 `(nil, false, nil)` —— 这是一次**正常的未命中**，不是违约。

这属于「假前置条件」：客户读注释会以为空 key 非法，于是不敢传， 或者传了之后以为行为未定义。两种误解都比不写更糟。

两条合法出口，必须选一条：

选择	动作	配套
空 key 合法	删掉前置条件；AF 说明空 byte 串也是合法键	RI 删掉 `len(...) >= 1`；补空 key 用例
空 key 非法	`Get` / `Set` / `Del` 检查 `len(key) == 0` 并返回哨兵错误	定义错误值；补拒绝用例

注意空的 `[]byte` 与 `nil` 在 `string(key)` 之后都是 `""`，所以这两种输入会命中同一个键。 合同要么把它们都当合法空键，要么都拒绝，不能只拒绝其中一种。

3.2.4. P0 二：`val == nil` 不能当作未命中判据

spec 写「未命中时 `val == nil`」，这半句是对的；但它暗示了反方向也成立，而探针的第三行直接推翻：`Set(k, nil)` 之后 `Get(k)` 返回 `val == nil, ok == true`。

原因是 `Set` 没有拒绝 nil 值，而 `bytes.Clone(nil)` 返回 nil。于是 `val` 有三种状态，`nil` 同时属于两种：

场景	VAL	OK	客户能怎么判断
键不存在	nil	false	只能看 `ok`
键存在，值为空切片	非 nil，长度 0	true	`len(val) == 0` 不足以区分
键存在，值为 nil	nil	true	只能看 `ok`

合同必须加一句：**`ok` 是唯一可靠的命中判据；`val == nil` 不表示未命中**。这不是抠字眼——客户如果写 `if val == nil { return notFound }`，在存过 nil 值的键上就会误判。

3.2.5. P0 三：没写返回值是否别名 rep

方法注释只说了「返回 val」，没说它是副本。实现第 70 行 `bytes.Clone` 明确是拷贝，探针也验证了改返回值不影响 store。这条必须写进合同，理由有两个：

1. 客户可能直接 `append` 返回值，如果将来实现改成别名，就会写坏 rep；
2. Safety 一节已经声明了「Get 做防御性拷贝」，方法注释不写，客户读方法时看不到。

标准库的句式可以直接借用：`ToLower returns a copy of the byte slice s ...`。

3.2.6. P1 三项

1. **摘要措辞**：`the new value` 里的 `new` 没有参照物，且偏实现。`sync.Map.Load` 的摘要直接把未命中写进去：`or nil if no value is present`。你也可以把摘要写成 `Get returns the value associated with key.`，下一段立刻说未命中。
2. **虚构的错误路径**：`Other condition` 指的是什么条件？实现没有任何 `return err` 分支，所以 `err` 恒为 nil。规格应当写 `Get never returns a non-nil error`，而不是留一个客户永远触发不到的分支。
3. **零值语义**：探针证明 `KV{}` 上 `Get` 安全返回 `(nil, false, nil)`。但 `Open` 的合同还没定，`Get` 也没说「零值可读」还是「必须先 Open」。按 Go 惯例（`sync.Mutex`、`http.Client` 都明说零值可用），只要行为是安全的，就值得写一句。

3.2.7. 与测试的一致性

`kv_test.go` 第 41、50 行都是 `_, ok, err = kv.Get(...)`，把 `val` 丢弃了。也就是说，spec 新写的「未命中 `val == nil`」条款**没有任何测试验收**。按七动作的顺序，规格写完要补一条能证明它的断言；但当前测试强度下可以先记进 `TODO(阶段末)`，不必立刻加测试。

另外第 50 行仍然是 `Get([]byte("xxx"))`，删除后查的是从未存在的键，这条假绿仍在。

3.2.8. 真实项目对照

样本	写法	对应本 SPEC 的哪条

`sync.Map.Load`	摘要里写 `or nil if no value is present`，用 `ok` 判存在	P0 二 + P1 摘要
`slices.Clone`	`The result preserves the nilness of s`	P0 二：nil 是合法值，要写清
`proto.Unmarshal`	`The provided message must be mutable`，且实现真的不检查	P0 一：真前置条件的样子
`json.Unmarshal`	非法入参返回 `InvalidUnmarshalError`	P1：真错误路径的样子

`proto.Unmarshal` 的前置条件是「实现不做检查」的合法例子，因为它把违约定义为未定义行为；而你这份 spec 的实现实际上**接受**空 key，所以不能类比它。

3.3. 陷阱与易错

⚠️ **陷阱预警 前置条件不是「我希望客户怎么做」，而是「实现不保证的部分」。** 如果实现默默接受空 key，写 `len(key) >= 1` 只是在制造误解。

⚠️ **陷阱预警 `val == nil` 不是未命中的同义词。** 存过 nil 值的键也返回 nil。唯一可靠的判据是 `ok`；这条必须写进合同，否则客户一定踩。

⚠️ **陷阱预警 不要承诺不可达的错误。** `err` 恒 nil 时写「其他情况返回错误」，客户会写一个永远不执行的分支，测试也覆盖不到。

⚠️ **陷阱预警 拼写错误也是合同瑕疵。** `unexpectated`、`descripe`、`fale`、`equal 1` 的双空格 会原样出现在 `go doc` 和 pkg.go.dev 上；评审时会被当成「没读过的注释」。

3.4. 关键点

📌 **核心要点** `Get` 的 spec 目前有 3 条 P0：删掉或兑现 `len(key) >= 1`；写明 `ok` 是唯一命中判据（nil 值也返回 `val == nil`）；写明返回值是副本。补完这三条，合同才和实现一致。

优先级	动作	判据
P0	决定空 key 合法还是拒绝	注释与实现一致
P0	补「`ok` 是唯一命中判据」	nil 值命中也能被正确判断
P0	补「返回值是副本」	与 Safety 和 `bytes.Clone` 一致
P1	改摘要措辞，把未命中提前	`go doc` 首行可读
P1	把虚构错误路径改成「不产生错误」	没有不可达承诺

P1	写零值语义	`KV{}` 上 `Get` 有明确说法
P2	修正拼写与空格	`go doc` 输出干净

3.5. 变式检验

变式 1：如果空 key 被判为非法，`Get` 该返回什么？要么返回哨兵错误（如 `ErrEmptyKey`），要么 panic。Go 的惯例是返回错误，且错误字符串小写、可被 `errors.Is` 判断。`Set` / `Del` 也要同步检查，否则会出现「能存不能查」的不对称。

变式 2：如果将来 `Set` 拒绝 nil 值，`val == nil` 能否重新表示未命中？仍然不能。空切片 `[]byte{}` 也是合法值，`Get` 会返回非 nil、长度 0 的切片；`ok` 仍是唯一判据。这条不会因为 `Set` 变严而消失。

变式 3：把返回值改成别名（不拷贝），合同要改哪几句？删掉「副本」，改成「返回值在下次修改前有效」；Safety 要说明客户修改会写坏 rep；测试要验证「改返回值会影响 store」——与现在正好相反。

变式 4：零值 `KV{}` 上 `Get` 安全，但 `Set` 会 panic。合同怎么写才不矛盾？`Get` 写「零值可读，返回未命中」；`Set` 写「必须先 `Open`，否则 panic」。两个方法的前置条件不同，这在 Go 里很常见，但不能笼统写「必须先 Open」。

3.6. 同类归类

问题类型	本 SPEC 的现状	标准库的成熟做法
未命中语义	`val == nil` 当判据	`sync.Map.Load` 用 `ok`
nil 值	未区分「未命中」与「值为 nil」	`slices.Clone` 明说保留 nilness
前置条件	写了但没执行	`proto.Unmarshal` 的 `must` 由未定义行为兜底
错误路径	虚构 `other condition`	`json.Unmarshal` 写明真实错误类型
别名	未写	`bytes.ToLower` 写 `returns a copy`

3.7. Go 语言机制补充

nil map 可读不可写。读 nil map 返回零值，`delete` 是 no-op，写入会 panic。所以 `KV{}` 上 `Get` / `Del` 安全，`Set` 不安全。合同要按方法分别写，不能一句「必须先 Open」带过。

`bytes.Clone(nil)` 返回 nil。这是 `Set(k, nil)` 之后 `Get(k)` 返回 `val == nil` 的直接原因。要改「未命中必为 nil」为「未命中必为 nil 且 `ok == false`」，而不是给 `val` 加更多含义。

`string([]byte{})` 与 `string(nil)` 都是空串。所以空 key 与 nil key 在 map 里是同一个键。要么统一允许，要么统一拒绝。

`go doc` 只显示注释。拼写错误、语法错误会原样出现在 pkg.go.dev 上；`go vet` 不会检查注释文字。提交前用 `go doc ./internal/db KV.Get` 看一眼。

6.031 对照。 本题对应 R06 规格：前置条件、后置条件、异常都属于合同；「未命中是正常返回值还是错误」是 R06 的典型判别。`Get` 选择了 `ok` 而非 error， 与 `sync.Map.Load` 一致。

3.8. 本次操作与当前状态

已做的文件操作：

1. 读取 `internal/db/kv.go` 第 60–73 行的 `Get` spec 与实现；
2. 读取 `internal/db/kv_test.go`，确认第 41、50 行丢弃 `val`、第 50 行仍查 `xxx`；
3. 在 `$TMPDIR/kvprobe` 写探针测试与 `overlay.json`，用 `-overlay` 注入运行，**未写入仓库**；
4. 追加本一级标题到 `note/2026-11-12.typ`；
5. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `go test -overlay=... ./internal/db/ -run TestProbeGetSpec -v -count=1` ⟶ `PASS`；
2. 零值 `KV{}` 上 `Get` ⟶ `val == nil`、`ok == false`、`err == nil`，不 panic；
3. 空 key `Get` ⟶ `ok == false`、`err == nil`，前置条件未被实现执行；
4. `Set(k, nil)` 后 `Get(k)` ⟶ `val == nil`、`ok == true`，证明 `val == nil` 不能表示未命中；
5. 修改入参 / 返回值后再次 `Get` ⟶ 仍是 `v1`，双向拷贝成立；
6. `typst compile 2026-11-12.typ` ⟶ 待编译确认（见下）。

需要你自己动手的：

1. P0：决定空 key 合法还是拒绝，并让注释与实现一致；
2. P0：在 spec 里补一句「`ok` 是唯一可靠的命中判据，`val == nil` 也可能表示命中」；
3. P0：补「返回值是副本」一句，与 Safety 的 `bytes.Clone` 对齐；
4. P1：把 `Other condition` 改成「`Get` 当前不产生非 nil 错误」；
5. P1：写清零值 `KV{}` 上 `Get` 的行为；
6. P2：修正 `unexpectated`、`descripe`、`fale` 与多余空格。

4. 为什么 spec 要写副作用，什么时候写

4.1. 费曼直觉

你说的应该是 **side effect（副作用）**，也就是 6.031 规格里的 `effects` 一栏。

先给定义：**副作用 = 函数在返回值之外，对世界造成的可观察改变。** 返回值是「函数告诉你的」，副作用是「函数背着你做的」。

类比药品说明书：`returns` 是「治什么病」，`effects` 是「会不会伤肝、能不能和酒一起吃」。只写疗效不写副作用，病人会按错误的方式吃药——代码也一样，只写返回值不写副作用，客户会在「能不能复用入参」「能不能并发」「失败了怎么办」上全猜错。

> 📌 **核心要点** 规格是客户唯一的推理依据。客户看不到实现，只能读合同。 副作用不写，客户就会假设「没有副作用」——而这是最危险的一种假设。

4.2. 硬核机制

4.2.1. 副作用有六种载体

载体	例子	客户会踩什么
入参变异	改 `[]byte`、map、指针指向的 struct	复用入参时数据已被改
receiver 状态	`KV.Set` 修改 `kv.mem`	两次调用结果不同；无法随意重试
返回值别名	返回内部切片、共享底层数组	改返回值写坏 rep
外部世界	文件、网络、stdout、环境变量、当前目录	副作用不可撤销；顺序敏感
资源与并发	打开句柄、启动 goroutine、持锁、定时器	泄漏、死锁、竞态
控制流	`panic`	客户没写 recover 就整个进程挂掉

Go 里最后一条常被漏掉：`panic` 不是错误，而是「不可恢复的副作用」。 标准库会在注释里明确写 `It panics if ...`，你也应该写。

4.2.2. 为什么必须写：四条推理链

客户想做的事	需要知道	没写会怎样
复用入参	函数有没有改它	传进去的缓冲区被悄悄改写
并发调用	函数有没有共享可变状态	数据竞争，`-race` 才抓得到
重试 / 幂等	第二次调用会不会累积效果	重试一次，数据被加了两遍
错误处理	失败时世界被改了多少	以为没写成功，实际写了一半

这四条都不是「文档美观」问题，而是**客户能否写出正确代码**的问题。而且它们无法从函数名和类型推出来：`Set` 看不出会不会拷贝，`Close` 看不出是否幂等，`Sort` 看不出会不会改入参。

4.2.3. 什么时候必须写：四问判据

调用一个函数之前，客户能观察到的「返回值之外的东西」，逐条问：

问题	回答「是」时	写什么
调用后，入参还能原样使用吗？	改入参	`modifies s in place` / `stores a copy of val`
调用后，对象自身的状态变了吗？	mutator	`mutates the store` / `subsequent calls see ...`
返回值与对象共享内存吗？	别名	`returns a copy` / `the result aliases ...`
失败时，世界被部分修改了吗？	非原子	`on error, the store is unchanged`
会影响进程级状态吗？	全局副作用	`changes the working directory` 等
会 panic 吗？	控制流副作用	`panics if the store is not open`

六个问题里只要有一个「是」，就必须在 `effects` 里写。全是「否」才是纯函数。

4.2.4. 什么时候可以不写

1. **纯函数且没有别名**：`bytes.Equal(a, b)` 只读两个入参，返回 bool，没有可观察改变。写一句摘要就够。
2. **实现内部的分配与缓存**：临时缓冲区、局部变量、不影响抽象状态的缓存，客户观察不到，不需要写。
3. **只读操作的并发声明**：Google Style Guide 说得很明确—— Go 用户默认认为只读操作并发安全，变异操作默认不安全；只有「不明确」或「API 自己提供同步」时才需要写并发条款。

但注意一个反例：「**返回副本**」**不是实现细节**。客户能观察到「改返回值不影响对象」，所以它属于合同，必须写。`bytes.ToLower` 就写了 `returns a copy`。

4.2.5. Go 的真实样本

样本一：修改入参，写在第一句。

```
1    // Unmarshal parses the JSON-encoded data and stores the result
2    // in the value pointed to by v. If v is nil or not a pointer,
3    // Unmarshal returns an [InvalidUnmarshalError].
```

Listing 21: GOROOT `src/encoding/json/decode.go`： stores the result 就是 effects

`stores the result in the value pointed to by v` 一句话交代了「改的是 v 指向的内存」。客户因此知道：v 必须是可写的，且调用后 v 的内容会变。

样本二：失败时也发生的副作用。

```
    // If the returned error is nil, the [Response] will contain a non-nil
    // Body which the user is expected to close. If the Body is not both
    // read to EOF and closed, the [Client]'s underlying [RoundTripper]
    // (typically [Transport]) may not be able to re-use a persistent TCP
    // connection to the server for a subsequent keep-alive request.
    //
    // The request Body, if non-nil, will be closed by the underlying
    // Transport, even on errors.
```

Listing 22: GOROOT `src/net/http/client.go` 的 `Client.Do`（节选）：资源副作用 + 失败语义

这段是副作用写作的范本，因为它同时回答了三个问题：

1. **谁负责清理**：返回的 Body 由调用者关闭；
2. **不清理的后果**：连接无法复用（不是「可能泄漏」这种空话）；
3. **失败时呢**：请求 Body 即使出错也会被关闭。

第三条最关键——客户最怕的正是「失败了，我的东西还在不在」。

样本三：禁止持有引用，而不是「会修改」。

```
    // Implementations must not retain p.
```

Listing 23: GOROOT `src/io/io.go`：反向副作用条款，约束实现者不得保留引用

副作用条款不只写「我做了什么」，也写「我承诺不做什么」。`must not retain p` 是后者：实现者不得把 p 存起来，客户因此可以复用缓冲区。

样本四：反例，变异没写。

```
    // Sort sorts a slice of any ordered type in ascending order.
    // When sorting floating-point numbers, NaNs are ordered before other
values.
    func Sort[S ~[]E, E cmp.Ordered](x S)
```

Listing 24: GOROOT `src/slices/sort.go`：实际就地修改 x，注释没写

对照 `bytes.ToLower` 的 `returns a copy`，就能看出缺的是什么。标准库自己也有遗漏，所以「别人没写」不能当作不写的理由。

样本五：全局副作用，靠函数名和包文档兜底。

```
    // Chdir changes the current working directory to the named directory.
    // If there is an error, it will be of type [*PathError].
```

Listing 25: GOROOT `src/os/file.go`：改的是进程级 cwd，影响所有 goroutine

`Chdir` 的副作用是全局的：它会影响同一进程里所有 goroutine 的相对路径解析。注释只写了错误类型，没写「这是进程级状态」——客户得从名字推断。这类副作用如果不在名字里，就更应该写进注释。

4.2.6. 失败语义是副作用的一部分

`(T, error)` 的 Go 函数，客户最常问的是：**err 非 nil 时，对象还能用吗？被改了多少？**

三种常见合同，必须选一条并写清：

合同	含义	例子
原子	失败时状态完全不变	事务、先校验后写入
尽力而为	失败时可能已部分修改	`Client.Do`：请求可能已发出
失败后不可用	对象进入终止状态	`os.File.Close` 之后的写操作

不写这条，客户就没法决定「失败了要不要重试」「重试会不会写两遍」。`Client.Do` 的注释之所以长，就是因为它在回答「失败后哪些东西已经发生」。

4.2.7. 映射回 `internal/db`

方法	副作用	当前 SPEC 写的
`Get`	无（只读 rep）；但返回副本	未写「副本」，属于漏写
`Set`	改 rep；不修改入参（`bytes.Clone`）	未写；`updated` 语义也未定
`Del`	键存在时改 rep；不存在时 no-op	未写重复删除的效果
`Open`	重建 rep，**清空已有数据**	未写；这是最容易被忽略的副作用
`Close`	当前没有副作用（no-op）	未写「关闭后仍可用」

四个必须回答的问题：

1. `Set` 之后，客户还能不能改 `val`？（能，因为拷贝了——这是给客户的保证）
2. `Set` 在 `KV{}` 上会怎样？（panic，属于控制流副作用，必须写）
3. `Open` 重复调用会不会清空？（会，这是有破坏性的副作用）
4. `Del` 删不存在的键会不会改变状态？（不会，no-op）

> ⚠ **陷阱预警 `Open` 的「清空」是最危险的副作用。** 客户如果写 `defer kv.Open()` 或重复初始化，已有数据会无声消失。这条必须写进 `Open` 的 effects，或者改实现为「已初始化则 no-op」。

4.3. 陷阱与易错

> ⚠ **陷阱预警 只写返回值，不写状态变化。** `Set` 的 `updated` 恒真就是因为合同没定义「状态变化了什么」，实现随便返回一个值都能过测试。

> ⚠ **陷阱预警 把「不修改入参」当默认。** Go 没有 `const` 参数、没有 `readonly` 切片；切片头是副本，底层数组共享。不写 `stores a copy`，客户就会假设别名。

⚠️ **陷阱预警 漏写 panic。** `Set` 在 nil map 上 panic，这是副作用；客户不写 recover 就挂进程。 标准库的惯例是 `It panics if ...` ，照抄即可。

⚠️ **陷阱预警 漏写失败时的部分修改。**「err 非 nil」不等于「什么都没发生」。 `Client.Do` 明说请求 Body 即使出错也会被关闭，就是因为这条容易被误解。

⚠️ **陷阱预警 把「返回副本」当实现细节。** 它决定客户能不能 `append` 、能不能缓存返回值， 是合同的一部分，必须写。

4.4. 关键点

📌 **核心要点** 副作用的判据只有一句：**调用之后，除了返回值，客户还能观察到什么？** 只要答案不是「什么都没有」，就必须写进 `effects` 。 Go 的 doc comment 没有 `effects:` 标签，用陈述句写在同一段即可。

情形	写不写	句式
修改入参	必须	`stores a copy of val` / `modifies s in place`
修改对象状态	必须	`mutates the store`
返回值别名	必须	`returns a copy`
外部 I/O / 全局状态	必须	`writes to w` / `changes the working directory`
panic	必须	`panics if ...`
失败时状态	必须	`on error, the store is unchanged`
纯函数、无别名	不必	摘要即可
内部临时分配	不必	客户观察不到

4.5. 变式检验

变式 1：`Get` 有副作用吗？ 没有——它只读 `mem` ，不修改任何状态。但它返回的是**副本**，这不是副作用， 而是「别名条款」，仍然必须写，因为客户能观察到「改返回值不影响对象」。

变式 2：`Set` 的副作用是什么？ 修改 rep（ `kv.mem` ）；不修改入参（ `bytes.Clone` ）；在 nil map 上 panic。 三条都要写，缺一条客户就会误用。

变式 3：如果 `Set` 改成不拷贝、直接存 `val` ，副作用条款怎么变？ 从 `stores a copy of val` 改成 `retains val; the caller must not modify it afterwards` 。 这是一条**客户义务**，属于 requires 与 effects 的交界，必须写清。

变式 4：`Close` 当前是 no-op，要不要写副作用？ 要写「没有副作用」：`Close is a no-op; the store remains usable after Close`。不写的话，客户会以为关闭后对象失效。沉默不是中性，而是暗示。

变式 5：失败时的部分修改，Go 里怎么测？ 构造一个会失败的调用（如 nil 入参），失败后检查对象状态是否与调用前一致。 这就是「原子性」测试；当前 `internal/db` 没有失败路径，阶段末再补。

4.6. 同类归类

函数类型	副作用	写法
纯函数	无	摘要 + 返回值即可，如 `bytes.Equal`
producer	无，但返回副本	`returns a copy`，如 `bytes.ToLower`
mutator	改 receiver / 入参	`mutates` / `modifies x in place`
I/O	外部世界	`writes to w` + 清理责任
生命周期	打开 / 关闭资源	`the caller must close` + 幂等性

真实项目的分歧也说明必须写清：`slices.Sort` 省略了变异，`bytes.ToLower` 写了副本；`os.File.Close` 不幂等，`sql.Rows.Close` 幂等。名字相同，副作用相反。

4.7. Go 语言机制补充

切片是描述符，不是数组。传 `[]byte` 给函数，复制的是「指针 + 长度 + 容量」，底层数组共享。所以「不修改入参」必须靠拷贝兑现，不能靠类型约束。`bytes.Clone` / `slices.Clone` 是标准做法。

map 是引用类型。把 map 传给函数不会复制哈希表，函数里的写入对调用方可见。`KV` 的 `mem` 是私有字段，所以副作用被限制在对象内部——这是封装的价值。

string 不可变。 `string([]byte)` 会复制字节，所以 map 的键不受调用方后续修改影响。这条要写进 Safety，解释「为什么 key 不用 Clone，val 要」。

`panic` 不是 error。 Go 的惯例是：可恢复的失败返回 `error`，不可恢复的编程错误才 panic。 `Set` 在 nil map 上 panic 属于后者，但合同仍要写，客户才知道要先 `Open`。

`go vet` 不检查注释。 副作用写没写、写得对不对，工具不会告诉你；只能靠测试（验证行为）和评审（验证合同与行为一致）。

6.031 对照。本题对应 R06 规格（effects 与前置 / 后置条件同属合同）、R07 设计规格（强 / 弱、确定性 / 欠定）、R08 不可变（为什么 `[]byte` 要拷贝）。Java 的 `void` 方法天然需要写 effects；Go 的多返回值让人误以为「返回值就够」， 这是本题最容易产生的错觉。

4.8. 本次操作与当前状态

已做的文件操作：

1. 读取 `software-construction` 的 `references/concept-atlas.md` 中 C05 / C06 相关条目；

2. 用 `go doc -src` 读取 `net/http.Client.Do`、`os.Open` 的原文注释；
3. 复用前几节已核对的 `json.Unmarshal`、`io.Reader`、`bytes.ToLower`、`slices.Sort` 样本；
4. 追加本一级标题到 `note/2026-11-12.typ`；
5. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `go doc -src net/http.Client.Do` → 确认「请求 Body 即使出错也会关闭」与「响应 Body 必须读尽并关闭」两条副作用条款；
2. `go doc -src os.Open` → 确认资源打开 + `*PathError` 的写法；
3. 前几节已验证的 `slices.Sort`（未写变异）、`bytes.ToLower`（写副本）、`io.Reader`（`must not retain p`）继续作为正反样本；
4. `typst compile 2026-11-12.typ` → 待编译确认（见下）。

需要你自己动手的：

1. 在 `Set` 的 spec 里补三条副作用：修改 rep、不修改入参、nil map 上 panic；
2. 在 `Open` 的 spec 里写清「重复调用会清空已有数据」；
3. 在 `Get` 的 spec 里补「返回副本」（性质上属于别名条款）；
4. 在 `Close` 的 spec 里明说「no-op，关闭后仍可用」或改实现；
5. 回答前一节留下的问题：空 key 合法还是拒绝。

5. Get spec 二轮评审 + go doc 的渲染语法

5.1. 费曼直觉

先给判定：**这一版的结构比上一版清楚，但有一个致命的结构错误和两处内容矛盾。**

结构上的进步是实打实的：删掉了假前置条件（`Preconditions: nothing`），补了 `val is a clone`，用 `Parameters` / `Preconditions` / `Postconditions` 分了段。但它现在像一封写好了却没贴地址的信：**注释和 `func` 之间多了一个空行**，Go 不认为它是 `Get` 的 doc comment，`go doc` 一个字都不显示。

第二个问题是子项写法：`-val`（连字符后没有空格）加上 5 个空格缩进，在 Go doc 里既不是列表项、也不是嵌套列表——Go doc 根本**不支持嵌套列表**。实测渲染时，三个子项被合并成一句话。

> 📌 **核心要点** 这一轮的两个 P0 都不是「措辞不好」，而是「文档没有按你期望的方式出现」：一个是整段消失，一个是子项合并成流水句。先修这两处，再谈条款内容。

5.2. 硬核机制

5.2.1. 实测一：当前注释完全不被 go doc 显示

```
$ go doc ./internal/db KV.Get
package db // import github.com/zz-cpp/db-go/internal/db

func (kv *KV) Get(key []byte) (val []byte, ok bool, err error)
```

Listing 26: 当前仓库的真实输出：只有签名，注释正文一个字都没有

原因是第 84 行的注释与第 86 行的 `func` 之间有一个空行（第 85 行）。Go 的 doc comment 必须**紧邻**声明，中间不能有空行或其他注释块。这和 2026-11-10 笔记里 `// var (...)` 块夹在类型注释与声明之间是同一类错误。

5.2.2. 实测二：去掉空行后，子项被合并

把同一份注释去掉空行，放进临时模块再 `go doc`：

```
1    Get returns the new value associated with key.
2
3    Parameters:
4      - key: the key to look up.
5
6    Preconditions:
7
8      - nothing.
9
10   Postconditions:
11
12     - If key exists in the store: -val is a clone of the value associated
with
13       the key. -ok is thrue. -err is nil.
14
15     - If key does not in the store: -val is nil. -ok is false. -err is nil.
16
17     - If key is nil: -val is nil. -ok is false. -err is a obj represents
the
18       reasons of error occuring.
```

Listing 27: `go doc` 的真实渲染：三个子项被并进父项段落，`-val` / `-ok` / `-err` 成了普通文本

`go.dev/doc/comment` 的原话是：**List items only contain paragraphs, not code blocks or nested lists.** 列表项只能装段落，不能装代码块，也不能嵌套列表。所以：

写法	被当成	结果
`  - key: ...`（两空格 + `-`）	列表项	正常列表
`    -val ...`（五空格 + `-v`）	列表项的续行	并进上一段的文字
`  - If key exists:` 下面再缩进子项	嵌套列表	不支持，子项被并进段落
`    code`（四空格以上、非列表）	代码块	等宽渲染

5.2.3. 实测三：`Preconditions:` 不是标题

Go 1.19 起，标题的语法是一行以 `#` 加一个空格开头，且前后有空行。`Preconditions:` 带冒号，只是普通段落文字，渲染时和正文没有区别。若要真标题，写成 `# Preconditions`（`#` 后必须有空格）。

顺带一提：Go 1.19 之前，标题是「单行、后面空行、且没有结尾标点」的隐式规则；加冒号正是当年**阻止**它变成标题的办法。现在两条规则并存，但 `#` 更明确。

5.2.4. 内容问题清单

原文	判定	问题

注释与 `func` 之间有空行	**P0**	`go doc` 完全不显示
`-val ...` 子项	**P0**	嵌套列表不支持，子项被合并；`-` 后还缺空格
`If key is nil: ... err is a obj represents the reasons of error occuring`	**P0**	与实现矛盾：nil key 返回 `(nil, false, nil)`
（未写）`ok` 是唯一命中判据	**P0**	`Set(k, nil)` 后 `val == nil` 且 `ok == true`
`Get returns the new value associated with key.`	P1	`new` 含糊；摘要没写未命中
`Parameters: - key: the key to look up.`	P1	解释显然信息；Style Guide 建议只写非显然部分
`thrue` / `does not in the store` / `a obj represents` / `occuring`	P2	拼写与语法错误会原样出现在 pkg.go.dev

其中「nil key 返回错误」这条最严重，因为它在三个地方同时错：

1. 实现没有 `error` 分支，`Get` 永远返回 nil error（上一轮探针已实测）；
2. `nil` 与空字节串经 `string(key)` 后都是 `""`，所以 nil key 与空 key 行为相同；
3. `Preconditions: nothing` 已经承诺「任何输入都合法」，与「nil key 出错」自相矛盾。

5.2.5. 已经修好的两处

1. **假前置条件**：上一版 `It requires that the length of key is more than or equal 1` 已经删掉，换成了 `Preconditions: nothing`。这与实现一致（空 key 可查）。
2. **副本条款**：`val is a clone of the value associated with the key` 补上了出向拷贝。措辞可以再向标准库靠拢（`returns a copy of ...`），但语义已经对。

5.2.6. `go doc` 的渲染语法全表

出处：`go.dev/doc/comment`（Go 1.19+ 的 `go/doc/comment` 包实现）。注意：`gofmt` 不是语法的定义者，它只负责**规范化**；语法由这个文档定义。

元素	语法	GOFMT 会做什么
段落	无缩进的连续非空行	保留换行，不 rewrap；合并重复空行
标题	`#` + 空格 + 文本，无缩进，前后空行	把旧式隐式标题改成 `#`
无序列表	`*` `+` `-` `•` 之一 + 空格，整段缩进	统一成两空格 + `-`
有序列表	`1.` 或 `1)` + 空格，整段缩进	统一成 `1.`
嵌套列表	不支持（列表项只含段落）	无
代码块	缩进行（tab 或空格）	统一为一个 tab，前后插空行
链接	`[Text]: URL` 定义 + `[Text]` 引用	把定义移到注释末尾

文档链接	`[Name]`、`[Name.Method]`、`[pkg.Name]`	无
弃用	`Deprecated: ...` 段落	无
不支持	Markdown 强调、HTML、表格、图片	无

5.2.7. 本机实测：gofmt 到底改什么

用 `$TMPDIR/fmttest/f.go` 做了三组对照，结论以本机 Go 1.27 为准：

```
// Foo does things:
//
//   * first
//   * second
func Foo() {}
```

Listing 28: 输入：两空格缩进 + `*` 标记

```
// Foo does things:
//
//   - first
//   - second
func Foo() {}
```

Listing 29: gofmt 输出：`*` 被换成 `-`，缩进保持两空格

有序列表同理，`1)` 会被换成 `1.`：

```
//   1) first   ->   //   1. first
//   2) second  ->   //   2. second
```

Listing 30: gofmt 统一有序列表标记为 `1.`

代码块则统一成一个 tab：

```
// Bar shows code:
//
//     x := 1
//     y := 2
```

Listing 31: gofmt 输出：代码块每行以一个 tab 开头（`//` 后紧跟制表符）

还有一个容易忽略的边界：**顶格的 `* first` 不是列表**。列表必须整段缩进；顶格的星号只是普通段落文字，gofmt 不会改它。这与 CommonMark 不同，是 Go doc 自己的规则。

5.2.8. gofmt 做什么、不做什么

做：

1. 把缩进的 `*` / `+` 列表统一成 `-`，两空格缩进，续行四空格；
2. 把 `1)` 统一成 `1.`；
3. 把代码块统一成一个 tab，并在前后插入空行；

4. 把旧式隐式标题（单行、无结尾标点）改成 `#` 标题；
5. 把链接目标定义移到注释末尾，方便发现未使用项；
6. 把连续反引号或单引号转成 Unicode 引号。

不做：

1. 不 rewrap 段落文本（保留手工换行，所以可以用「一句一行」）；
2. 不检查拼写，也不检查条款与实现是否一致；
3. 不把嵌套列表变成合法结构（它根本不支持）；
4. **只在 doc comment 上生效**：注释必须无缩进、从第 1 列开始、紧邻下一个 token。当前 `Get` 的注释与 `func` 之间有空行，所以 gofmt 连处理都不处理。

最后一条可以用 `gofmt -d internal/db/kv.go` 验证：**没有任何输出**。不是格式正确，而是这段注释没被识别为 doc comment。

5.2.9. 建议的修改顺序（只给问题，不代写）

1. 删掉注释与 `func (kv *KV) Get` 之间的空行；
2. 把三个「If …」改成一级列表项，子项内容并进同一行；例如 `If key exists, val is a copy, ok is true, and err is nil.`；
3. 删掉整段 `If key is nil`（除非你打算让实现真的返回错误，并补测试）；
4. 补一句「`ok` 是唯一命中判据，存过 nil 值也会让 `val == nil`」；
5. 摘要改成 `Get returns ...`，把未命中放到第二段；
6. 决定空 key 的最终归属（这条决定了 `Preconditions` 到底是 `nothing` 还是别的）。

5.3. 陷阱与易错

⚠ **陷阱预警 注释与声明之间的空行，会让整段规格消失。**这是本轮最贵的错误：内容写得再全，`go doc` 里也看不到。改完注释第一件事就是跑一次 `go doc`。

⚠ **陷阱预警 Go doc 不支持嵌套列表。**子项不会变成子列表，而是被并进父项段落。需要表达「条件 → 三个结论」时，写成一句话，或把条件作为标题、结论作为一级列表。

⚠ **陷阱预警 `-` 后面必须有空格。** `-val` 不是列表标记，只是一段以连字符开头的文字。gofmt 不会替你补空格，它只重排已经合法的列表。

⚠ **陷阱预警 `Preconditions:` 不是标题。**标题必须是 `#` + 空格。带冒号的行按段落渲染，看起来和正文一样。

⚠ **陷阱预警 gofmt 不检查内容。**它能统一列表标记，但不会发现「nil key 返回 error」与实现矛盾，也不会抓拼写。渲染正确 ≠ 规格正确。

5.4. 关键点

> 📌 **核心要点** 这版 Get spec 的两个 P0 是渲染层面的：空行让注释消失，子项写法让内容合并。修完这两处再改条款。内容上剩一个 P0：`ok` 是唯一命中判据；以及一条与实现矛盾的 `If key is nil` 分支（实现不产生错误，应删）。

优先级	动作	验收
P0	删除注释与 `func` 之间的空行	`go doc ./internal/db KV.Get` 显示正文
P0	把子项改成一级列表或合并成句	渲染后不再出现 `-val` 连写
P0	删除或实现 `If key is nil` 的错误分支	注释与实现一致
P0	补「`ok` 是唯一命中判据」	nil 值命中能被正确判断
P1	摘要与措辞	`go doc` 首行可读
P2	拼写	`go doc` 输出干净

5.5. 变式检验

变式 1：如果坚持要「条件 + 三个结论」的结构，Go doc 里怎么写？ 把条件写成段落或 `#` 标题，三个结论写成一级列表：

```
// If key is present:
//
//   - val is a copy of the stored value
//   - ok is true
//   - err is nil
//
// If key is absent:
//
//   - val is nil
//   - ok is false
//   - err is nil
```

Listing 32: 条件用段落、结论用一级列表：三段都合法，渲染不会合并

变式 2：`go doc` 和 pkg.go.dev 的渲染一样吗？ 同一套 `go/doc/comment` 语法，pkg.go.dev 会额外把 `[Name]` 渲染成链接、把 `Deprecated:` 标成弃用样式。命令行 `go doc` 只输出纯文本结构。

变式 3：`gofmt` 会把 `*` 列表改成 `-`，为什么我的没变？ 先检查列表有没有整段缩进。顶格的 `* first` 是段落，不是列表；gofmt 只对识别为列表的 span 做规范化。

变式 4：注释里写 Markdown 的 `*bold*` 会怎样？ 原样显示，不会加粗。Go doc 不支持字体变化；唯一支持的强调是 `_italic_`？—— 不，连斜体也不支持（`go/doc/comment` 有 `Italic` 类型，但只能由解析器内部产生，注释语法里没有对应的标记）。所以 `*bold*` 会带星号出现在文档里。

变式 5：为什么 `gofmt -d` 对当前文件没有输出？因为注释与 `func` 之间有空行，不是 doc comment，gofmt 的注释重排根本不会触发。 这正好是「空行导致注释消失」的第二个证据。

5.6. 同类归类

体系	层级结构	本项目对应
Go doc	段落 + `#` 标题 + 一级列表 + 代码块	当前 Get 注释要改成的形状
Markdown	可嵌套列表、强调、表格	Go doc 不支持，别照搬
Javadoc	`@param` / `@return` 标签	Go 没有标签，用段落
pkg.go.dev	Go doc 语法的 HTML 渲染	部署后的最终外观

真实项目的注释都遵守同一形状：`sync.Map.Load` 是一段散文， `os.File.Close` 是三句散文， `io.Reader` 是多段 + 一个列表都没有。 Go 生态更偏爱短段落，而不是多层列表。

5.7. Go 语言机制补充

doc comment 的关联规则。 注释必须紧邻声明：无空行、无其他注释块、 从第 1 列开始。`go/doc` 只把这样的注释当作 doc comment；否则它只是普通注释，`go doc` 不显示，`gofmt` 也不重排。

`go/doc/comment` 是解析器，不是渲染器。 `go doc` 与 pkg.go.dev 都用它把注释 解析成 `Doc` / `Block` / `Text`，再按各自的方式打印。语法定义在 `go.dev/doc/comment`，实现位于 `/usr/lib/go/src/go/doc/comment/` 。

`gofmt` 的注释重排入口。 `/usr/lib/go/src/go/printer/comment.go` 的 `formatDocComment` 会调用 `comment.Printer.Comment`；触发条件是 `printer.go` 第 743–749 行：注释无缩进、紧邻下一个 token、且前一个 token 不是 `import` 。 这就是「空行会导致不重排」的源码级原因。

列表必须缩进。 `/usr/lib/go/src/go/doc/comment/print.go` 第 227–248 行打印列表时， 每项输出一个前导空格 + `-` （数字列表是 `1.` ），续行四个空格。解析器只把「缩进的 span」当作列表，顶格标记不算。

`go vet` 不检查注释。 拼写、空行、嵌套列表它都不管； `go doc` 是唯一的验收工具，pkg.go.dev 是最终展示。

6.031 对照。 本题对应 R06 规格（合同写法）与 C03 代码评审（注释可读性）。 6.031 的 Javadoc 有 `@param` / `@return` 标签，Go 没有；但「前置 / 后置 / 副作用」 三件事仍然要写，只是改用段落和列表表达。

5.8. 本次操作与当前状态

已做的文件操作：

1. 读取 `internal/db/kv.go` 第 60–91 行的 Get spec 与实现；
2. 用 `go doc ./internal/db KV.Get` 验证当前渲染结果（只有签名）；
3. 在 `$TMPDIR/kvfmt` 复制 `kv.go` 、删除空行后重新 `go doc` ，验证子项合并；

4. 在 `$TMPDIR/fmttest` 写了三组注释，实测 `gofmt` 对 `*` / `1)` / 代码块的规范化；
5. 读取 GOROOT 的 `go/printer/comment.go`、`go/printer/printer.go`、`go/doc/comment/print.go`，确认触发条件与列表打印实现；
6. 抓取并阅读 `go.dev/doc/comment` 的语法章节；
7. 追加本一级标题到 `note/2026-11-12.typ`；
8. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `go doc ./internal/db KV.Get` → 只输出签名，注释未显示；
2. 去掉空行后 `go doc` → 显示正文，且三个子项被合并为一句；
3. `gofmt -d internal/db/kv.go` → 无输出（注释未被识别为 doc comment）；
4. `gofmt` 对两空格缩进的 `*` 列表 → 输出 `-`；
5. `gofmt` 对 `1)` 列表 → 输出 `1.`；
6. `gofmt` 对顶格 `*` → 不改（不是列表）；
7. `gofmt` 对四空格代码块 → 统一为一个 tab；
8. `typst compile 2026-11-12.typ` → 待编译确认（见下）。

需要你自己动手的：

1. P0：删掉 Get 注释与 `func` 之间的空行，再跑一次 `go doc`；
2. P0：把嵌套子项改成一級列表或合并成句（Go doc 不支持嵌套）；
3. P0：删掉 `If key is nil` 的错误分支，或改实现并补测试；
4. P0：补「`ok` 是唯一命中判据」；
5. P1：摘要改成 `Get returns ...`，把未命中放到第二段；
6. P2：修正 `thrue`、`does not in the store`、`a obj represents`、`occuring`。

6. 0101 spec 完成度盘点与下一步

6.1. 费曼直觉

先给结论：**地基和一面墙有了，离能住还差得远。**

按可核对的合同条目算，0101 一共有 9 项： 5 个方法 spec + AF + RI + Safety + testing strategy。 现在**有内容的只有 1 项**（`Get`），而且它还有 2 个 P0；**没有任何一项达到「可判定、与实现一致」的标准。**

所以完成度是：

1. 有内容：1 / 9；
2. 达标（无 P0、与实现一致）：0 / 9；
3. 测试：绿，但只覆盖冒烟路径，且第 50 行是假绿。

> 📌 **核心要点** 测试绿 ≠ 合同完成。当前状态是 Apply 通过、Explain / Predict / Discriminate 都没有证据。「测试通过」只能证明现有断言没被抓到，不能证明规格写对了。

6.2. 硬核机制

6.2.1. 完成度总表

条目	状态	缺口
`Get`	有内容，格式已合规	仍缺 `ok` 唯一判据；`If key is nil` 与实现矛盾
`Set`	只有 4 个 TODO	`updated` 语义、同键同值、空键 / 空值、拷贝政策
`Del`	只有 3 个 TODO	`deleted` 语义、键不存在、重复删除
`Open`	只有 3 个 TODO	零值可用性、重复调用是否清空、error 条件
`Close`	只有 3 个 TODO	关闭后能否用、是否幂等、error 条件
AF	一行集合式 + TODO	抽象域、`string` 键与字节键的对应、空 map 的含义
RI	两行 + TODO	不可判真假；与 `Set` 行为矛盾；缺 `mem != nil`
Safety	一段 + TODO	入向 / 出向未分离；`key` 为何不拷贝；未命中返回什么
testing strategy	三个空 TODO	Set 分区、history 分区、未覆盖子域清单

6.2.2. 验证基线（刚跑过）

```
$ gofmt -l internal/db/
internal/db/kv_test.go

$ go build ./... && go vet ./... && go test ./internal/db/ -count=1
ok  github.com/zz-cpp/db-go/internal/db  0.003s
```

Listing 33: 当前基线：测试绿，但 `kv_test.go` 还没过 gofmt

`gofmt -d internal/db/kv_test.go` 的差异只有一处：块注释结尾少了缩进空格， `*/` 应为 `*/` 。属于 P2，但 `gofmt -l` 有输出就不算干净。

6.2.3. 三个已经做对的

1. **Get 的结构**： `Preconditions` / `Postconditions` 分段清楚，列表是合法的一级列表；
2. **假前置条件已删**： `len(key) >= 1` 换成了 `nothing` ，与实现一致；
3. **副本条款**： `val is a clone of ...` 写进了合同，Safety 也有了落点。

6.2.4. 三个最大缺口

缺口	为什么贵	后果
4 / 5 个方法没有 spec	客户只能读实现	测试无法验收语义，实现怎么变都「对」
RI 与实现矛盾	注释说键值非空， `Set` 却接受空键空值	读者信注释，客户踩坑
`Set` 的 `updated` 恒真	布尔值退化常量	同键同值的语义永远无法被验证

6.2.5. 依赖链：为什么下一步必须先定「空键 / 空值」

这一个决策同时卡住四个地方：

受影响的位置	怎么受影响
RI	两条 `len(...) >= 1` 保留还是删除
`Get`	`If key is nil` 分支是返回未命中还是报错
`Set` / `Del`	要不要在写入 / 删除前检查长度并返回错误
testing strategy	边界分区里要不要加空键 / 空值用例

所以顺序不能反过来。先写 `Set` 的注释、再去改 RI，会改两遍。

6.2.6. 两种选择的连锁后果

选择	要改的地方	不用改的地方
空键空值合法	删 RI 两行； `Get` 的 nil 分支改成未命中；AF 说明空串也是合法键	实现完全不动；测试可以后补

空键空值非法	`Set` / `Del` / `Get` 加检查；定义哨兵错误；RI 保留并补 `mem != nil`	实现要改，测试要补拒绝用例

两条都合法。判据是：**你想要「任何字节串都是键」的简单模型，还是「键必须非空」的防御性模型？** 前者实现最简单，也最接近 `sync.Map`；后者需要错误值和三处检查。

> ⚠️ **陷阱预警 不要选第三种：注释写「非法」、实现默默接受。** 这正是当前 RI 的状态，也是评审里最常被判「写了不维护的不变量」的情形。

6.2.7. 下一步的完整顺序

顺序	动作	验收
1	回答空键 / 空值决策；改 RI 两行	`go doc ./internal/db KV` 显示可判谓词
2	修 `Get` 的 nil 分支 + 补 `ok` 唯一判据	`go doc` 正文与实现一致
3	写 `Set` 的 spec；定 `updated` 语义	每个返回值都有说法
4	写 `Del` 的 spec	重复删除有明确结果
5	写 `Open` / `Close` 的 spec	零值、重复调用、关闭后都有说法
6	填 AF 三句 + Safety 分向	AF 能回答「空 map 是什么」
7	改 testing strategy 三个 TODO；第 50 行改 `k1`	没有空 `TODO`；删除有验收
8	跑 `gofmt` / `build` / `vet` / `test` / `go doc`	全绿且 `gofmt -l` 无输出

其中第 3 步可能牵动实现：如果 `updated` 定义为「值是否变化」，当前「先写后比」恒真的实现要改成「先读旧值、比较、再写」。这一步按七动作应当先写测试看到红，再改实现。

6.2.8. 真实项目参照

你要写的	参照样本	可借的句式
`Get` 的 `ok`	`sync.Map.Load`	`ok` 判断是否存在，未命中不返回 error
`Set` 的 `updated`	`sync.Map.Store`（不返回布尔）	若布尔无信息量，考虑删掉它
`Close` 的两条路线	`os.File.Close` / `sql.Rows.Close`	不幂等且失效 / 幂等且可用
空键的取舍	`sync.Map` 接受任意 `any` 键	简单模型：不限制键

`sync.Map` 的 `Store` 没有返回值，说明「写成功与否」在内存 map 上通常不值得用布尔表达。如果你的 `updated` 只是恒真的装饰，删掉它比定义它更诚实——但接口是测试定的，先别动。

6.3. 陷阱与易错

⚠️ **陷阱预警 先写实现再补规格，会写两遍。** `Set` 的 `updated` 一旦按「写后比较」实现，再定「值是否变化」的语义时就要改实现和测试。规格先行不是形式主义。

⚠️ **陷阱预警 空 TODO 不是计划。** `partition on history: TODO` 让读者分不清「知道缺」还是「忘了写」。阶段末清单必须写成 `TODO(阶段末): 缺 X、Y 。`

⚠️ **陷阱预警 `gofmt -l` 有输出就是没做完。**当前 `kv_test.go` 的块注释结尾差一个空格；这类问题和规格无关，但会让「验证全绿」这句话不成立。

⚠️ **陷阱预警 测试绿会掩盖 `val` 没被断言。** 第 41、50 行都是 `_, ok, err = kv.Get(...)`，`val` 被丢弃，所以 `Get` 的 nil 条款至今没有验收。

6.4. 关键点

📌 **核心要点** 0101 的合同完成度是 1 / 9（有内容），0 / 9（达标）。下一步不是继续润色 `Get`，而是先做「空键 / 空值」决策——它同时决定 RI、`Get` 的 nil 分支、`Set` / `Del` 的边界和测试分区。

里程碑	完成判据	当前
Apply	测试绿、build / vet 通过	已满足
Explain	每个返回值都能说出语义	1 / 5 方法
Predict	能预测空键、nil 值、重复删除、关闭后	0 项有合同依据
Discriminate	未命中 vs 错误、nil vs 空切片、幂等 vs 非幂等	未写

6.5. 变式检验

变式 1：如果空键非法，`Get([]byte{})` 返回什么？哨兵错误（如 `ErrEmptyKey`），并且 `Set` / `Del` 同样拒绝。`nil` 与 `[]byte{}` 经 `string()` 后都是空串，必须一起处理。

变式 2：如果空键合法，RI 还剩什么？至少 `mem != nil`（除非零值可写），以及「键与值都是任意字节串」。原来的 `len(...) >= 1` 全部删除，AF 要说明空串也是合法键。

变式 3：`updated` 如果保留，最小有意义的定义是什么？「值是否发生变化」：新键 true、同键不同值 true、同键同值 false。这要求实现先读旧值再比较，不能写后比较。

变式 4：为什么不能先补测试再定规格？测试断言必须来自合同。合同没定就写测试，等于把某个实现的行为固化成规格，以后改实现会与测试冲突——这正是「改测试迁就实现」的来源。

6.6. 同类归类

层次	当前状态	完成标志
合同文本	9 项中 1 项有内容	9 项都能 `go doc` 读到
合同与实现一致	RI、`Get` 的 nil 分支、`updated` 三处不一致	没有注释说假话
测试验收	冒烟 + 一处假绿	关键条款都有断言
文档渲染	Get 已合规	全部方法 `go doc` 可读

对照 6.031 的分层：现在只到 Apply，因为 Explain 需要「每个返回值都有语义」，而 4 / 5 个方法连一句话都没有。

6.7. Go 语言机制补充

`gofmt -l` 的语义。 它列出「需要格式化」的文件；无输出才算干净。`gofmt -d` 显示具体差异。两者都不改文件，`-w` 才写回。

`go doc` 是规格的验收工具。注释写得对不对，`go doc <pkg> <symbol>` 和 `pkg.go.dev` 是唯一能看到最终效果的地方；`go vet` 不检查注释内容。

零值语义要逐个方法写。`KV{}` 上 `Get` / `Del` 安全，`Set` 会 panic。这是同一类型的两个方法有不同前置条件的典型例子，不能笼统写「必须先 Open」。

`error` 与哨兵错误。如果决定拒绝空键，定义 `var ErrEmptyKey = errors.New(...)`，用 `errors.Is` 判断；错误字符串小写、不带标点（Go 惯例）。

6.031 对照。本题对应 R06 规格与 R11 AF / RI。R11 要求 RI 对每个合法 rep 都能判真假——当前 `len( each of mem.keys) >= 1` 连语法都不是 Go，更谈不上判定。

6.8. 本次操作与当前状态

已做的文件操作：

1. 读取 `internal/db/kv.go` 全文（120 行）、`internal/db/kv_test.go` 全文（54 行）；
2. 运行 `gofmt -l`、`go build`、`go vet`、`go test` 确认基线；
3. 运行 `gofmt -d internal/db/kv_test.go` 定位唯一的格式差异；
4. 追加本一级标题到 `note/2026-11-12.typ`；
5. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `gofmt -l internal/db/` → 输出 `internal/db/kv_test.go`（块注释结尾缩进）；
2. `go build ./...` → 通过；
3. `go vet ./...` → 无输出；
4. `go test ./internal/db/ -count=1` → `ok`；
5. `gofmt -d internal/db/kv_test.go` → 仅 `*/` 应为 `  */` 一处差异；
6. `typst compile 2026-11-12.typ` → 待编译确认（见下）。

需要你自己动手的：

1. 第 1 步（今天）：回答「空键 / 空值合法还是拒绝」，并据此改 RI 两行；
2. 第 2 步：修 `Get` 的 nil 分支，补 `ok` 唯一判据；
3. 第 3–5 步：写 `Set` / `Del` / `Open` / `Close` 的 spec（先规格，后实现）；
4. 第 6 步：填 AF / Safety；
5. 第 7 步：改 strategy 与第 50 行；
6. 第 8 步：`gofmt -w internal/db/kv_test.go` 后跑完整验证。

7. Set 的 spec 评审：三个矛盾与一个恒真布尔

7.1. 费曼直觉

先给判定：**比 Get 进了一步，但注释和实现在三处各说各话。**

进步是实打实的：加了 `ErrKeyIsNil`、加了 nil 检查、写了 Preconditions / Postconditions。但它现在的状态像「门口挂着『禁止吸烟』，屋里却在发烟」：注释说空切片键会返回 `false`，实现却返回 `true`；注释说 `Preconditions: nothing`，实现却在零值上 panic；`updated` 这个返回值，无论怎么调都是 `true`。

> 📌 **核心要点** 评审 Set 只需要做一件事：把注释里的每个「If ...」逐条跑一遍。三个分支里有两个与实现不符，一个（`updated` 的语义）根本没定义。

7.2. 硬核机制

7.2.1. 先看渲染：注释又没显示

```
$ go doc ./internal/db KV.Set
package db // import github.com/zz-cpp/db-go/internal/db

func (kv *KV) Set(key []byte, val []byte) (updated bool, err error)
```

Listing 34: 当前输出只有签名：注释与 `func` 之间又有一个空行

这和 Get 上一轮的错误完全一样。**改任何 Set 的内容之前，先删掉这个空行**，否则所有修改都不会出现在 `go doc` 里。

7.2.2. spec 承诺 vs 实测

探针通过 `-overlay` 注入，不写仓库，覆盖七种输入：

```
new key:            updated=true err=<nil>
same key same val:  updated=true err=<nil>
same key diff val:  updated=true err=<nil>
empty non-nil key:  updated=true err=<nil>
nil key:            updated=false errIsNil=true
nil value:          updated=true err=<nil>
zero KV Set:        panic=true
inbound copy:       stored=v1
```

Listing 35: Set 的实际行为（`go test -overlay=... -run TestProbeSetSpec -v`）

输入	SPEC 承诺	实测	判定
新键	`updated true`，`err nil`	`true`，`nil`	一致
同键同值	（未定义）	`true`，`nil`	缺
同键不同值	（未定义）	`true`，`nil`	缺
空切片键 `[]byte{}`	`updated false`，`err nil`	`true`，`nil`	**矛盾**

nil 键	`updated false`，`ErrKeyIsNil`	`false`，`ErrKeyIsNil`	一致
nil 值	（未定义）	`true`，`nil`	缺
零值 `KV{}`	`Preconditions: nothing`	`panic`	**矛盾**
调用后改入参	（未写）	store 不变	缺条款

7.2.3. 矛盾一：空切片键

spec 写的是：

```
// If len(key) is 0, but != nil:
//   - updated is false.
//   - err is nil.
```

Listing 36: spec 声称空切片键返回 `(false, nil)`

实现只拦截 `key == nil`，`[]byte{}` 不是 nil，于是走正常写入路径，`updated` 由「写后比较」得到 true。所以 spec 与实现相反。

更麻烦的是键的同一性：`string(nil)` 和 `string([]byte{})` 都是 `""`。于是出现这个不对称：

```
kv.Set(nil, v)        -> (false, ErrKeyIsNil)   // 被拒
kv.Set([]byte{}, v)   -> (true, nil)            // 成功，写入键「空串」
kv.Get(nil)           -> 读到上面写的 v，ok == true
```

Listing 37: 同一个键（空串），在 Set 里被区别对待，在 Get 里又不区分

这不是「两种输入两种语义」，而是**同一个键在三个方法里三种规矩**。必须统一：要么 `len(key) == 0` 一律拒绝（nil 与空切片都拒），要么全接受，把 `ErrKeyIsNil` 删掉。只检查 `key == nil` 是两者之间最差的选项。

7.2.4. 矛盾二：`Preconditions: nothing` 与零值 panic

spec 说前置条件是 `nothing`。实测 `KV{}.Set(k, v)` 直接 panic：`kv.mem` 是 nil map，写 nil map 会 `panic: assignment to entry in nil map`。

前置条件写 `nothing` 等于承诺「任何 `KV` 都能 Set」，与事实相反。要么写「必须先 `Open`」，要么让 `Set` 在 nil map 时返回错误（会更像 Go 的风格）。

7.2.5. 矛盾三：`updated` 是一个恒真布尔

三个「成功」分支实测全是 true：

1. 新键 → true；
2. 同键同值 → true；
3. 同键不同值 → true。

原因是实现先写、再读刚写入的克隆、再比较：

```
    kv.mem[string(key)] = bytes.Clone(val)
    newVal, _ := kv.mem[string(key)]
    updated = bytes.Equal(val, newVal)
```

Listing 38: 写后比较：`newVal` 是 `val` 的克隆，`bytes.Equal` 必然为 true

`newVal, _` 的 comma-ok 也是多余的：刚写入的键必然存在。 这个布尔值现在不携带任何信息，客户只能忽略它。

两种出口：

选择	语义	实现
保留 `updated`	值是否变化：新键 true、不同值 true、同值 false	先读旧值、比较、再写
删掉 `updated`	改为只返回 `error`	参考 `sync.Map.Store` 不返回布尔

接口由测试决定，先别动签名；但**合同必须选一条**，否则 `updated` 永远恒真。

7.2.6. 缺失的条款

1. **副作用**：`Set` 修改 rep；不修改入参（`bytes.Clone` 已实测）；零值上报 panic。 三条现在一条都没写。
2. **空值**：`Set(k, nil)` 成功，之后 `Get(k)` 返回 `val == nil, ok == true`。这条直接影响 Get 的判据，Set 的合同里应该说明「nil 是合法值」。
3. **同键同值**：是覆盖、是 no-op、还是返回 false？spec 完全没提。

7.2.7. 格式与实现细节

```
    // Preconditions:
    //   - noting.
```

Listing 39: 列表用 tab 缩进；去掉空行后 gofmt 会把它改成两空格 + `-`

当前 `gofmt -l internal/db/` 只报 `kv_test.go`，不报 `kv.go` —— **不是因为 kv.go 格式对，而是因为注释没被识别为 doc comment**。 在临时副本上删掉空行后再跑 `gofmt -d`，就会看到它把 `//\t- noting.` 规范成 `//  - noting.`。这是「空行导致 gofmt 也不处理」的第二个证据。

另外 `Postconditions: the key and val will be a stored unique pair.` 里的 `unique pair` 含义不明：是说键唯一，还是说键值对唯一？这句话无法判真假。

7.2.8. 真实项目对照

样本	返回值设计	对本 Set 的启示
`sync.Map.Store`	不返回布尔	没有信息量就不返回
`sync.Map.LoadOrStore`	返回 `loaded bool`，表示键是否已存在	布尔要有明确所指

bbolt `Bucket.Put`	只返回 `error`	写入成功用不返回错误表达
`map` 赋值	无返回值	覆盖与否由客户自己查旧值

`LoadOrStore` 是「有意义的布尔」的样板：`loaded` 明确表示「之前是否存在」。你的 `updated` 如果继续存在，就应该选一个这样明确的含义。

7.3. 陷阱与易错

⚠️ **陷阱预警 注释与 `func` 之间的空行，Again。** 这一轮 Set 的注释又没显示。删空行是改 Set 的第一步，也是验收的第一步。

⚠️ **陷阱预警 `key == nil` 不是「空键」的完整检查。** `[]byte{}` 长度 0 但非 nil，会绕过检查写入空串键；而 `string(nil)` 与 `string([]byte{})` 是同一个键 `""`。要拒绝就检查 `len(key) == 0`，不要只检查 nil。

⚠️ **陷阱预警 `Preconditions: nothing` 是一句强承诺。** 零值 panic 直接推翻它。写「nothing」之前，先跑一次零值调用。

⚠️ **陷阱预警 写后比较必然为 true。** 用刚写入的值和入参比较，等于拿克隆和原件比。要让 `updated` 有意义，必须先读旧值。

7.4. 关键点

📌 **核心要点** Set 的 spec 有 3 个 P0：① 空切片键的承诺与实现相反；② `Preconditions: nothing` 与零值 panic 矛盾；③ `updated` 语义未定义、实现恒真。修完这三条，再补副作用与 nil 值条款。

优先级	动作	验收
P0	删除注释与 `func` 之间的空行	`go doc ./internal/db KV.Set` 显示正文
P0	统一空键规则：`len(key) == 0` 全拒或全接受	`Set(nil)` 与 `Set([]byte{})` 行为一致
P0	写清零值语义，或让 `Set` 返回错误	没有「承诺 nothing 却 panic」
P0	定义 `updated` 语义并改实现	同键同值的行为可被断言
P1	补副作用：改 rep、不改入参、panic	三条都有句子
P1	补 nil 值条款	`Set(k, nil)` 后的 `Get` 行为有依据
P2	修正 `seted` / `setted` / `noting` / `a obj`	`go doc` 输出干净

7.5. 变式检验

变式 1：如果统一拒绝空键，`Get` 要不要也跟着拒绝？要。否则 `Set` 拒绝、`Get` 却把 nil 当空串去查，仍然是三种规矩。`Del` 同理。三个方法对键的合法性判断必须一致。

变式 2：如果 `updated` 定义为「值是否变化」，实现怎么改？先取旧值和 `existed`，再写入，最后 `updated = !existed || !bytes.Equal(old, val)`。注意 nil 与空切片的比较：`bytes.Equal(nil, []byte{})` 为 true，要决定这是否算变化。

变式 3：如果删掉 `updated`，测试要改哪里？`kv_test.go` 第 35 行 `updated, err := kv.Set(...)` 要改成只接 `err`；这是接口变更，需先与测试的使用方对齐，不要顺手改。

变式 4：零值 `KV{}` 上 `Set` 应该 panic 还是返回错误？Go 的惯例是：可恢复的调用错误返回 `error`，编程错误才 panic。「忘了 `Open`」更像编程错误，panic 可接受，但必须在 `Preconditions` 里写明。如果选择返回错误，则要定义哨兵错误并在 `Set` 里检查 `kv.mem == nil`。

7.6. 同类归类

决策点	本 SET 的现状	标准库的成熟做法
空键	只拒 nil，空切片放行	内置 map 允许 `""`；要拒就拒 `len == 0`
零值可用性	`Preconditions: nothing` 但 panic	`sync.Mutex` 明说零值可用
布尔返回值	恒真	`LoadOrStore` 的 `loaded` 有明确含义
入参拷贝	已拷贝，未写进合同	`bytes.ToLower` 写 `returns a copy`
nil 值	接受，未写	`slices.Clone` 明说保留 nilness

7.7. Go 语言机制补充

nil slice 与空 slice。`var k []byte` 是 nil，`[]byte{}` 是非 nil 空切片；`len` 都是 0，`string()` 都是 `""`。所以它们在 map 里是同一个键。检查「空键」应该用 `len(key) == 0`，检查「nil 入参」才用 `key == nil`。两者语义不同，不能混用。

写 nil map 会 panic。`var m map[string][]byte; m["k"] = v` 触发 `panic: assignment to entry in nil map`。读 nil map 和 `delete` 是安全的。`KV{}` 的 `mem` 是 nil，所以 `Set` panic、`Get` / `Del` 不 panic。

`bytes.Clone(nil)` 返回 nil。所以 `Set(k, nil)` 存进去的是 nil，之后 `Get(k)` 返回 nil 且 `ok == true`。这不算 bug，但必须在合同里写。

`errors.New` 与哨兵错误。`ErrKeyIsNil` 已导出，用 `errors.Is(err, ErrKeyIsNil)` 判断。错误字符串应小写、无标点，`key is nil` 符合惯例。

6.031 对照。 本题对应 R06 规格（前置 / 后置 / 异常）与 R07 设计规格 （返回值的确定性）。`updated` 恒真属于「规格欠定到无法判定」的典型。

7.8. 本次操作与当前状态

已做的文件操作：

1. 读取 `internal/db/kv.go` 的 Set 注释 (第 97–118 行)、实现 (第 120–128 行)、 `ErrKeyIsNil` 定义（第 34 行）；
2. 在 `$TMPDIR/setprobe` 写探针测试与 overlay，验证七种输入的实际行为；
3. 在 `$TMPDIR/setfmt` 复制 `kv.go`、删除空行后跑 `gofmt -d`，验证 tab 缩进会被规范化；
4. 追加本一级标题到 `note/2026-11-12.typ`；
5. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `go doc ./internal/db KV.Set` → 只显示签名（空行导致注释未关联）；
2. 探针：新键 / 同键同值 / 同键不同值 / 空切片键 / nil 值全部 `updated=true`；
3. 探针：nil 键 → `false, ErrKeyIsNil`；
4. 探针：零值 `KV{}` 上 `Set` → panic；
5. 探针：修改入参后 store 仍为 `v1` （入向拷贝成立）；
6. `gofmt -l internal/db/` → 只报 `kv_test.go` （kv.go 因空行未被处理）；
7. 临时副本删空行后 `gofmt -d` → 把 `//\t- noting.` 改成 `//  - noting.`；
8. `typst compile 2026-11-12.typ` → 待编译确认（见下）。

需要你自己动手的：

1. P0：删掉 Set 注释与 `func` 之间的空行；
2. P0：统一空键规则——把 `key == nil` 改成 `len(key) == 0` (全拒)，或删掉检查 (全接受)；
3. P0：写清零值语义（「必须先 `Open` 」或「返回错误」）；
4. P0：定义 `updated`，并按定义改实现（先读旧值再比较）；
5. P1：补副作用三条与 nil 值条款；
6. P2：修正 `seted` / `setted` / `noting` / `a obj represents`。

8. Postcondition 与 effect 的区别

8.1. 费曼直觉

先澄清术语：你说的 `efficient` 应该是 **effect（effects，副作用）**。这里按「postcondition 与 effect」回答。

一句话区别：

1. **postcondition（后置条件）**回答「函数交回什么」：返回值、最终状态、错误；
2. **effect（副作用）**回答「函数顺手改了什么」：入参、对象状态、文件、网络、全局状态。

类比：函数是一次快递。postcondition 是「你签收的包裹长什么样、里面是什么」；effect 是「快递员顺手改了你家门口的门牌号、把旧箱子带走了」。 两件事都可能在你签收后成立，但一个是关于「交回来的东西」，一个是关于「世界被改成了什么样」。

> 📌 **核心要点** 严格说，**postcondition 是父概念，effect 是它的一部分**： 后置条件 = 返回值 + 最终状态。 `effects` 这一栏专门写「最终状态里被改变的那部分」。所以问「有什么区别」，准确答案是：范围不同，不是并列关系。

8.2. 硬核机制

8.2.1. 两栏各自回答什么

栏位	回答的问题	典型句子
postcondition	函数返回时，返回值与可观察结果满足什么？	`returns a copy of s` / `ok is true`
effect	函数返回时，哪些状态被改变了？	`modifies s in place` / `writes to w`
两者的交集	状态改变后的可观察结果	`after Set, Get returns val`

注意最后一行：很多句子同时是两者。 `after Set, Get returns val` 既是后置条件（描述最终可观察结果），也是在描述 effect 的后果。区别不在句子，而在**你把它放在哪一栏、想强调什么**。

8.2.2. 包含关系：postcondition ⊇ effect

6.031 R06 把 postcondition 定义为「函数返回时保证为真的事情」， 它包含三块：

组成	内容	在 Go 注释里
returns	返回值的性质	第一段陈述句
effects	状态改变（入参 / receiver / 外部世界）	`modifies` / `writes to` / `stores`
throws / errors	错误或 panic 的条件	`returns ErrX if ...` / `panics if ...`

所以 `effects` 不是 postcondition 的对立面，而是它的一个子集。一个纯函数没有 effects，但仍然有 postcondition（例如 `returns true iff a and b are equal`）。

8.2.3. 四个样本拆解

函数	POSTCONDITION	EFFECT
`bytes.ToLower`	返回副本，字母映射为小写	无；不修改 `s`
`slices.Sort`	返回后 `x` 升序	修改 `x` 本身
`os.Chdir`	成功时 cwd 变成 `dir`	改进程级 cwd，影响所有 goroutine
`Set(k, v)`	返回 `updated`；之后 `Get(k)` 能读到 `v`	修改 rep（`kv.mem`）

四个里只有 `bytes.ToLower` 是「有 postcondition、无 effect」。这正好说明两者不能互相替代：只写 postcondition 会漏掉「`x` 被改了」，只写 effect 会漏掉「返回什么」。

8.2.4. 一个容易站错队的条款：别名

`Get returns a copy of the value` 属于哪一栏？

1. 它约束的是**返回值**与 rep 的关系，所以写进 postcondition 更准确；
2. 它也在防止 rep exposure，所以 6.031 会在 Safety 里再强调一次。

结论：**主栏是 postcondition，Safety 做交叉引用。** 不要把「返回副本」写进 effects——effects 描述的是「状态被改」，而「返回副本」描述的是「返回值不共享内存」，两者方向相反。

同理，`io.Reader` 的 `Implementations must not retain p` 是一条 **实现者义务**，它约束的是「不要把入参存起来」，属于 effects 的否定形式（「不产生某种状态改变」）。Go 的写法里常和 effect 条款放在一起。

8.2.5. 为什么容易混

1. 两者都以「函数返回之后」为真，时间点相同；
2. Go 的 doc comment 没有 `effects:` 标签，只能写成散文，边界靠作者自己划；
3. 很多函数的效果和返回值互相定义：`Set` 的 `updated` 就是「效果发生了吗」的返回值；
4. 6.031 的 `effects` 栏有时把「返回值」也塞进去，导致学生以为 effects = 后置条件。

> ⚠ **陷阱预警 不要把 effects 当作 postcondition 的同义词。** effects 只覆盖「状态改变」；把返回值条款也塞进 effects，会让读者分不清「函数返回了什么」和「世界变成了什么」。

8.2.6. 在 Get / Set 的 spec 里怎么落地

你的条款	归属	理由
`val is a clone of ...`	postcondition	描述返回值与 rep 的关系
`ok is true / false`	postcondition	描述返回值

`err is nil`	postcondition / errors	描述错误条件
（应写）`Set` 修改 rep	effect	状态改变
（应写）`Set` 不修改入参	effect（否定形式）	承诺不产生某种状态改变
（应写）零值上 panic	effect / errors	控制流也是可观察后果

当前 Set 的 `Postconditions: the key and val will be a stored unique pair.` 实际上是一条 **effect**（修改了 rep），却被放在 Postconditions 标题下—— 这不算错（effect 是 postcondition 的子集），但它没有说清「返回值 `updated` 是什么」，读者会把「存进去了」误当成「返回值一定是 true」。

8.2.7. 真实项目样本

```
    // Unmarshal parses the JSON-encoded data and stores the result
    // in the value pointed to by v. If v is nil or not a pointer,
    // Unmarshal returns an [InvalidUnmarshalError].
```

Listing 40: `json.Unmarshal`：第一句是 effect（stores the result），第二句是 postcondition（返回什么错误）

```
    // ToLower returns a copy of the byte slice s with all Unicode letters
mapped to
    // their lower case.
```

Listing 41: `bytes.ToLower`：只有 postcondition，没有 effect——它不改 `s`

```
    // Close closes the [File], rendering it unusable for I/O.
    // On files that support [File.SetDeadline], any pending I/O operations
will
    // be canceled and return immediately with an [ErrClosed] error.
    // Close will return an error if it has already been called.
```

Listing 42: `os.File.Close`：effect（关闭、取消 I/O）与 postcondition（重复调用返回错误）分成两句

三个样本的共同形状是：**先把「交回什么」和「改了什么」分开说**，读者不需要猜哪句属于哪栏。

8.3. 陷阱与易错

⚠️ **陷阱预警 effects 不等于 postcondition。** 前者是后者的子集（状态改变部分）。把两者等同，会漏写返回值条款。

⚠️ **陷阱预警 把「返回副本」写成 effect。** 它说的是返回值不共享内存，属于 postcondition；effects 描述的是「状态被改」，方向相反。

⚠️ **陷阱预警 只写 effect 不写返回值。**「存进去了」不等于「`updated` 为 true」。Set 当前的 `updated` 恒真，正是这种混淆的后果。

⚠️ **陷阱预警 只写 postcondition 不写 effect。** `slices.Sort` 就是现成反例：后置条件写全了，读者仍然不知道 `x` 被改了。

8.4. 关键点

📌 **核心要点** postcondition 回答「函数交回什么」，effect 回答「函数改了什么」。前者是父概念，后者是它的子集；一个纯函数可以有 postcondition 而没有 effect，但没有 effect 的函数依然需要写后置条件。

你想说的事	写进哪栏	句式
返回值是什么	postcondition	`returns ...`
返回值与 rep 的关系	postcondition + Safety	`returns a copy`
入参被改了	effect	`modifies s in place`
对象状态被改了	effect	`mutates the store`
外部世界被改了	effect	`writes to w` / `changes the working directory`
失败时状态如何	effect + errors	`on error, the store is unchanged`
会 panic 吗	effect / errors	`panics if ...`

8.5. 变式检验

变式 1：`bytes.ToLower` 有 effect 吗？没有。它返回新切片，不修改 `s`。`returns a copy` 是 postcondition，不是「修改了 s」的 effect。

变式 2：`slices.Sort` 的 postcondition 是什么？effect 是什么？postcondition：返回后 `x` 升序；effect：`x` 被就地修改。缺 effect 就会让持有同一切片的客户踩坑。

变式 3：`Set` 的 `updated` 属于哪一栏？返回值，所以是 postcondition。但如果它想表达「状态是否变化」，那它同时是对 effect 的总结。这正是它容易失控的原因。

变式 4：如果函数返回 error，失败时的状态该写在哪？写在 effect 里，用「失败时状态不变」或「失败时可能部分修改」表达。`Client.Do` 的「请求 Body 即使出错也会关闭」就是这种条款。

变式 5：纯函数需要写哪一栏？只需要 postcondition（并且通常没有 effects 栏）。但若返回值与入参共享内存（返回切片的一部分），仍要写别名条款。

8.6. 同类归类

概念	回答的问题	本项目落点

requires	调用者必须先满足什么	`Preconditions: nothing`
postcondition	函数交回什么、最终状态如何	`Get` 的三个 If 分支
effect	状态被改了什么	Set 修改 rep、Open 清空数据
returns	返回值的性质	`val is a clone` / `ok is true`
errors / throws	何时失败	`ErrKeyIsNil`

8.7. Go 语言机制补充

Go 没有 `effects:` 标签。 所有这些都写成 doc comment 里的陈述句。 `gofmt` 只重排格式，不检查你写的是 effect 还是 postcondition。

多返回值让边界更模糊。 `Set` 同时返回 `updated` 和 `err`， 读者容易把「存进去了」当成后置条件的全部，而忘记 `updated` 的语义需要单独定义。

`(T, error)` 的失败语义属于 effect。 err 非 nil 时对象是否还能用、状态是否被部分修改，这些都是可观察的状态改变，必须写进 effect 条款。

6.031 对照。 R06 的 postcondition 包括返回值与副作用；R07 讲规格的强弱与确定性。本题的区别在 R06 的「postcondition 由 returns + effects 组成」。

8.8. 本次操作与当前状态

已做的文件操作:

1. 读取 `software-construction` 的 `references/concept-atlas.md` 中 C05 规格条目（项目证据写作 `requires / effects / returns / throws`）；
2. 复用前几节已核对的 `json.Unmarshal`、 `bytes.ToLower`、 `os.File.Close`、 `slices.Sort` 样本；
3. 追加本一级标题到 `note/2026-11-12.typ`；
4. 未修改 `internal/db` 下任何代码或测试文件。

验证结果:

1. 本节为概念辨析，未新增命令验证；引用的注释原文均来自前几节的 `go doc -src` 输出与本机模块缓存；
2. `typst compile 2026-11-12.typ` → 待编译确认（见下）。

需要你自己动手的:

1. 在 Set 的 spec 里把「修改 rep」明确写成 effect（可以放在 Postconditions 下， 但要在句子里写清是状态改变）；
2. 把 `updated` 的语义单独定义成 postcondition（返回值条款）；
3. 给 Get 补一条「不修改 store」的 effect 条款（否定形式），避免读者猜测。

9. 忽略 Open / Close 后的 spec 全面检查（Get / Set / Del / AF / RI / Safety）

9.1. 费曼直觉

先给排序：**Del 是三者里最完整的，Set 卡在格式和一个恒真布尔，Get 有一处跨方法不一致， 类型注释（AF / RI / Safety）是最大的空白。**

用房子打比方：

1. `Del` 的墙砌好了，只是瓷砖贴歪了（内容对，渲染成代码块）；
2. `Set` 的墙砌了一半，门口还挡着一块布（注释不显示）；
3. `Get` 的三面墙里有一面和邻居家的不一样高（nil 键在 Get 和 Set / Del 里行为不同）；
4. AF / RI / Safety 还只是脚手架（公式、两行伪代码、一段散文），没浇混凝土。

> 📌 **核心要点** 按 skill 的 C05 / C07 / C09 判据，当前最贵的不是「再润色措辞」，而是 **RI 说假话**（值长度约束被实现违反）和 **nil 策略三处不一致**。 这两条会让测试无论怎么加都验不到真合同。

9.2. 硬核机制

9.2.1. 实测证据（通过 -overlay 注入，未写仓库）

```
delete existing:   deleted=true err=<nil>
delete again:      deleted=false err=<nil>
delete absent:     deleted=false err=<nil>
delete nil key:    deleted=false errIsNil=true
delete empty key:  deleted=false err=<nil>
zero KV Del:       panic=false

Set empty key:     updated=false err=<nil>
Get(nil) after:    val=[] ok=false err=<nil>
nil value hit:     val==nil=true ok=true err=<nil>
```

Listing 43: Del / Set / Get 的边界行为实测（Go 1.27，本机）

9.2.2. 逐项问题清单

位置	级别	问题
`Set` 注释	**P0**	注释与 `func` 之间有空行，`go doc KV.Set` 只显示签名
`Get` 的 nil 分支	**P0**	spec 说 `err` 非 nil，实现返回 `nil,false,nil`；Set / Del 却返回 `ErrKeyIsNil`
`Get` 缺判据	**P0**	nil 值命中时 `val == nil` 且 `ok == true`， `ok` 是唯一判据未写
RI 值约束	**P0**	`len(mem.values) >= 1` 被 `Set(k, nil)` 违反（实测成功）

RI 语法	**P0**	`mem.keys` / `mem.values` 不是 Go，也无法判真假
RI 缺 `mem != nil`	**P0**	零值 `Set` panic（上一轮实测）；零值 `Del` 不 panic
`Safety`	**P0**	`mem.val` 不是合法表达；入向 / 出向未分离；未写 key 为何不拷贝
`Del` 渲染	**P1**	Postconditions / Side Effects 用 tab 缩进，整段被渲染成代码块
`Del` 参数说明	**P1**	`the key to look up` 是从 Get 复制的，删除操作应为「要删除的键」
`Del` 的 Side Effects	**P1**	`removed from memory` 不准确；未写「键不存在时无副作用」
`Set` 的 `updated`	**P1**	成功路径恒真；同键同值算不算更新仍未定义
AF	**P1**	`TODO` 未填：抽象域、`string` 键与字节键的对应、空 map 的含义
三个 `Preconditions: nothing`	**P1**	nil 键会返回错误，与「nothing」的强承诺需要一句协调
`checkRI`	**P1**	仍是注释掉的空壳，没有调用点
testing strategy	**P1**	三个空 `TODO`；第 50 行仍查 `xxx`；新分支无测试
拼写 / 语法	P2	`thrue`、`seted` / `setted`、`noting`、`a obj represents`、`occuring`

9.2.3. P0 展开

一、Set 的注释又不显示。

```
// If key == nil:
//   - updated is false.
//   - err is obj represents the key is nil.

func (kv *KV) Set(key []byte, val []byte) (updated bool, err error) {
```

Listing 44: 注释与 `func` 之间的空行：`go doc KV.Set` 只输出签名

二、nil 键在三个方法里三种规矩。

输入	Set	Del	Get
`nil`	`false, ErrKeyIsNil`	`false, ErrKeyIsNil`	`nil, false, nil`
`[]byte{}`	`false, nil`	`false, nil`	`nil, false, nil`
非空键	写入，`true, nil`	删除，`true/false, nil`	查询，`val, ok, nil`

Set 和 Del 已经把 nil 当成错误，Get 却把它当普通未命中。 Get 的 spec 写的是「nil → err 非 nil」，实现没有这一支—— 所以这一处**同时**是「spec 与实现不一致」和「三个方法策略不一致」。

三、RI 在说假话。

```
    len( each of mem.keys) >= 1
    len( each of mem.values) >= 1
```

Listing 45: 当前 RI：键约束现在由 Set 维护，值约束完全没维护

键约束：`Set` 新增了 `len(key) == 0` 分支，空键不会被写入，所以「键非空」暂时成立。值约束：`Set(k, nil)` 与 `Set(k, []byte{})` 都会成功写入，值为 nil 或长度 0， 直接违反 `len(mem.values) >= 1` 。实测：

```
    nil value hit:     val==nil=true ok=true err=<nil>
```

Listing 46: 存 nil 值后 Get 命中，说明 RI 的值约束没有被任何路径维护

两条路：要么 `Set` 拒绝空值（并定义错误、补测试）， 要么删掉值约束、在 AF 里承认「值可以是任意字节串，含 nil」。

四、Safety 需要拆成三句。

现在只有一句：

```
    mem is private
    mem.val is mutable, So Get() and Set() make defensive copies to avoid
sharing the rep's values in mem with clients
```

Listing 47: 当前 Safety：方向对，但 `mem.val` 不是合法表达，也没有分向

缺的是：

方向	要写的内容
入向	`Set` 存 `bytes.Clone(val)` ； `key` 经 `string(key)` 复制字节，所以不需要额外拷贝
出向	`Get` 返回 `bytes.Clone(val)` ，客户修改返回值不影响 rep
未命中	`bytes.Clone(nil)` 返回 nil，客户不能用 `val == nil` 判断未命中
`Del`	不返回 rep 内部引用，只返回布尔和错误

9.2.4. 已经做对的三处

1. **Del 的分支与实现完全一致**：存在 → `true, nil` ；重复删除 / 不存在 → `false, nil` ；nil → `false, ErrKeyIsNil` 。实测逐条对上。
2. **Del 有 Side Effects 栏**：方向正确，只是措辞和渲染要修。
3. **Set 的空键处理现在自洽**： `nil` 走错误分支， `[]byte{}` 走 `false, nil` ， 与 spec 的分支一致（上一轮的空切片矛盾已修）。

9.2.5. 按 skill 的质量门槛对照

门槛（C05 / C07 / C09）	当前状态	判定
规格与测试一致	spec 新增 nil / 空键分支，测试一条都没覆盖	不通过
RI 可检查	`mem.keys` 不是 Go 表达式	不通过
AF 是全函数	公式只覆盖非空 map，空 map 的含义未写	不通过
rep exposure 路径全部封堵	入向 / 出向只写了一句，`Del` 未提	不通过
没有为了绿改测试	测试没改，但已落后于 spec	部分
未命中 vs 错误	Get 用 `ok`，Set / Del 用 `ErrKeyIsNil`，策略不统一	不通过

9.2.6. 三步修复顺序（忽略 Open / Close）

顺序	动作	验收
1	删 Set 注释与 `func` 之间的空行	`go doc KV.Set` 显示正文
2	统一 nil 策略：Get 也返回 `ErrKeyIsNil`，或 Set / Del 改回未命中	三个方法对 nil 行为一致
3	修 RI：值约束二选一 + `mem != nil` + 改成 Go 可判表达式	`go doc KV` 里的 RI 能逐条判真假

第 2 步是真正的分叉点：**nil 键到底算错误还是算普通键？** 选「错误」就三处都返回 `ErrKeyIsNil`（Get 要加检查）；选「普通键」就三处都当空串处理（Set / Del 要删掉检查）。现在选哪种都行，但不能三种规矩并存。

9.3. 陷阱与易错

⚠ **陷阱预警 RI 的两行不是「注释」，是合同。** 值约束写了却不维护，比不写更糟：读者会假设空值不可能出现，而 `Set(k, nil)` 就能造出来。

⚠ **陷阱预警 `Set` 的注释空行，Again and again。** 每次改完注释先跑 `go doc`，否则你改的内容只存在于源码里，不在文档里。

⚠ **陷阱预警 `mem.val` 不是 Go 的写法。** map 的值没有 `.val`；写 `mem[k]` 或「map 的值」。这类伪代码会让读者怀疑 RI 是从别处抄来的。

⚠ **陷阱预警 tab 缩进会把整段变成代码块。** `Del` 的 Postconditions 在 `go doc` 里是等宽块而不是列表。列表要用「两空格 + `-` 」，且整段作为列表 span。

9.4. 关键点

> 📌 **核心要点** 忽略 Open / Close 后，P0 有七项：Set 注释不显示、Get 的 nil 分支、Get 缺 `ok` 判据、RI 值约束、RI 语法、RI 缺 `mem != nil` 、Safety 不完整。 其中「nil 策略三处不一致」和「RI 说假话」是根，先修这两个。

对象	最强的一处	最弱的一处
`Get`	格式已合规，分支清楚	nil 分支与实现矛盾
`Set`	空键处理自洽	注释不显示； `updated` 恒真
`Del`	三个分支与实现完全一致	渲染成代码块；参数说明复制错
`KV` 类型	三段结构齐全	RI 不可判定且与实现矛盾

9.5. 变式检验

变式 1：如果 nil 键统一算错误，Get 要加什么？ 在 `Get` 开头加 `if key == nil { return nil, false, ErrKeyIsNil }`，spec 的 `Preconditions` 改成「key 不得为 nil」，并补测试。 这样三个方法对 nil 的行为一致。

变式 2：如果 nil 键统一算普通键，Set / Del 要改什么？ 删掉两处 `if key == nil` 检查；`Set(nil)` 写入空串键， `Del(nil)` 删除空串键； spec 的三个 nil 分支全部删掉，AF 说明空串是合法键。

变式 3：如果值必须非空， `Set` 要怎么改？ 在写入前检查 `len(val) == 0`，返回哨兵错误（如 `ErrEmptyValue` ）； RI 保留值约束；补空值测试。注意 `nil` 与 `[]byte{}` 都是长度 0，要一起拒绝。

变式 4： `updated` 保留，最小有意义的定义是什么？「值是否变化」: 新键 true、不同值 true、同值 false。实现要先读旧值再比较。如果不想改实现，就删掉这个返回值，只留 `error` 。

变式 5：Del 的 Postconditions 怎么排版才不被当代码块？ 用「两空格缩进 + `-` 」的一级列表，条件写成段落；或把三个分支合并成两句话。 当前 tab 缩进 + `-` 的写法会被识别为代码块。

9.6. 同类归类

问题类型	本项目现状	标准库的做法
nil 键	三个方法三种规矩	内置 map 允许 `""` ；要拒就统一拒
值可为 nil	RI 说不行，实现可以	`slices.Clone` 明说保留 nilness
布尔返回值	`updated` 恒真	`LoadOrStore` 的 `loaded` 有明确含义
RI 写法	伪代码	`sync.Mutex` 的「零值可用」是可判定的散文
Side Effects	Del 有， Get / Set 缺	`Client.Do` 把资源副作用写全

9.7. Go 语言机制补充

nil map 的读写差异。 读 nil map 返回零值、`delete` 是 no-op、写入 panic。 所以零值 `Get` / `Del` 安全，`Set` 不安全。RI 里写 `mem != nil` 或者 在 `Set` 里检查并返回错误，二选一。

`string(nil)` 与 `string([]byte{})` 都是空串。这是「nil 键与空键是同一个键」的根源。检查 nil 用 `key == nil` ，检查空键用 `len(key) == 0`；两者语义不同。

`bytes.Clone(nil)` 返回 nil。 所以 nil 值是合法可存储的， `Get` 命中时 `val == nil` 与未命中无法靠 `val` 区分，只能靠 `ok` 。

`go doc` 的列表与代码块。两空格 + `-` 是列表；tab 或四空格以上的缩进是代码块。`Del` 现在用 tab，所以整段被渲染成 preformatted text。

6.031 对照。 本题对应 R11 AF / RI（RI 必须可判、AF 必须全覆盖）、 R08 不可变与别名（Safety 的分向拷贝）、R06 规格（nil 是错误还是正常返回值）。

9.8. 本次操作与当前状态

已做的文件操作:

1. 读取 `internal/db/kv.go` 全文（168 行），重点 Get / Set / Del 与 KV 类型注释；
2. 用 `go doc ./internal/db KV` 、 `KV.Set` 、 `KV.Del` 检查渲染；
3. 在 `$TMPDIR/delprobe` 写探针与 overlay，验证 Del / Set / Get 的边界行为；
4. 读取 `software-construction` 的 concept-atlas 中 C07（别名）、C09（AF/RI）条目；
5. 追加本一级标题到 `note/2026-11-12.typ`；
6. 未修改 `internal/db` 下任何代码或测试文件。

验证结果:

1. `go doc ./internal/db KV.Set` → 只显示签名（空行）；
2. `go doc ./internal/db KV.Del` → 显示，但 Postconditions / Side Effects 是代码块；
3. `go doc ./internal/db KV` → AF / RI / Safety 均以代码块形式显示；
4. 探针：Del 存在 → `true, nil`； 重复删除 → `false, nil`； 不存在 → `false, nil`；
5. 探针：Del nil → `false, ErrKeyIsNil`； Del 空键 → `false, nil`； 零值 Del 不 panic；
6. 探针： `Set([]byte{})` → `false, nil` （不写入）；
7. 探针： `Get(nil)` → `nil, false, nil` （与 Get spec 矛盾）；
8. 探针： `Set(k, nil)` 后 `Get(k)` → `val == nil, ok == true` （违反 RI 的值约束）；
9. `typst compile 2026-11-12.typ` → 待编译确认（见下）。

需要你自己动手的:

1. P0：删 Set 注释与 `func` 之间的空行；
2. P0：统一 nil 策略（Get 加检查，或 Set / Del 删检查）；
3. P0：在 Get 里补「`ok` 是唯一命中判据」；
4. P0：修 RI：值约束二选一、补 `mem != nil` 、改成 Go 可判表达式；
5. P0：Safety 拆成入向 / 出向 / 未命中 / Del 四句；

6. P1：Del 的列表改成两空格缩进，参数说明改成「要删除的键」；
7. P1：定义 `updated` 或删掉它；
8. P1：补 strategy 的三个 `TODO`，第 50 行改 `k1`；
9. P2：统一拼写。

10. 验收策略校准：当前测试强度下哪些要做、哪些推迟

10.1. 费曼直觉

你的原则是：作者把项目拆成 45 步、测试逐步添加，所以规格只需满足**当前**测试强度；等某模块的测试全部提供完，再补残留缺陷。

这个原则我接受。但要加一条底线：

> 📌 **核心要点 合同可以说「未定」，不能说「假的」。**「同键同值的行为待定」可以留 `TODO(阶段末)`；「`len(key) == 0` 返回 `false`」但实现返回 `true`，不能留——那不是「未来会补」，是「现在文档在骗人」。

打个比方：装修按当前图纸验收，没做的房间可以空着；但已经挂上「承重墙」牌子的那面墙，不能其实是空心砖。 空着是进度问题，挂牌骗人是安全问题。

按这个底线，之前的清单要重新分三类：

1. **现在做**：改注释让它与实现一致、修当前测试里的假绿、修文档渲染；
2. **阶段末**：扩大测试范围或改实现语义的事，全部推到作者测试补全后；
3. **不做**：并发、持久化、后续 step 的接口。

10.2. 硬核机制

10.2.1. 当前测试到底约束了什么

`TestKVBasic` 一共 7 条断言：

```
Open            -> err == nil
Set(k1, v1)     -> updated && err == nil
Get(k1)         -> val == v1 && ok && err == nil
Get(xxx)        -> !ok && err == nil
Del(xxx)        -> !deleted && err == nil
Del(k1)         -> deleted && err == nil
Get(xxx)        -> !ok && err == nil
```

Listing 48: 当前测试的七条断言：只覆盖基本路径与「未命中」

它**没有**约束：nil 键、空切片键、nil 值、同键同值、零值、`Close` 后、别名拷贝。 所以这些维度的规格，按你的原则都属于「未定」，不必现在写成强承诺。

10.2.2. 三类事项的重新分级

类别	事项	为什么
现在做	`Set` 注释与 `func` 之间的空行	否则 `go doc` 看不到 spec
现在做	`Get` 的 `If key is nil` 分支改成与实现一致	实现返回 `nil,false,nil`，注释说返回错误
现在做	`Get` 补「`ok` 是唯一命中判据」	合同缺关键判据，容易误用

现在做	RI 删掉值长度约束（或改述为「值可为任意字节串」）	实现允许 nil 值，RI 说不行
现在做	RI 补 `mem != nil`（由 `Open` 建立）	零值 `Set` 会 panic
现在做	Safety 拆成入向 / 出向 / 未命中三句	纯注释，不扩大测试
现在做	`Del` 的列表改用两空格缩进	当前被渲染成代码块
现在做	`Del` 的参数说明改成「要删除的键」	从 Get 复制来的错误说明
现在做	第 50 行 `xxx` 改成 `k1`	当前测试的假绿，不属于「新测试」
现在做	`gofmt -w internal/db/kv_test.go`	`gofmt -l` 有输出
阶段末	nil 键 / 空切片键策略统一	当前测试不测；三方法不一致先记 TODO
阶段末	空值 / nil 值语义	当前测试不测
阶段末	`updated` 同键同值的定义（及实现改动）	当前测试只要求新键 true
阶段末	`Open` / `Close` 生命周期	当前测试只调用一次
阶段末	零值 `KV{}` 的可用性	当前测试先 `Open`
阶段末	`checkRI` 的实现与调用点	属于新增检查
阶段末	AF 的抽象域细化	可以现在补一句，也可留到阶段末
不做	并发安全、持久化、后续 step 接口	不属于 0101

10.2.3. 「阶段末」必须有触发条件

否则 `TODO(阶段末)` 会变成永远不还的债。建议在 strategy 里写清：

```
TODO(阶段末): 等作者给出本模块（KV 基本操作）的全部测试后，再补以下子域：
  - nil 键 / 空切片键
  - nil 值 / 空值
  - Set 同键同值
  - Open / Close 生命周期与零值
  - Safety 入向 / 出向拷贝
```

Listing 49: 阶段末清单：写清「等什么」和「补什么」，而不是一个空 TODO

触发条件就是你说的那句：**该模块的测试作者提供完了**。在此之前，测试强度是唯一的验收标准；在此之后，清单变成待办。

10.2.4. 我之前超纲的要求（收回）

按你的原则，下面这些我上一轮列成 P0/P1 的要求应当降级：

之前的要求	现在的处置
统一 nil 策略（Get 加检查或 Set/Del 删检查）	降级为阶段末；现在只让 Get 的注释与实现一致
拒绝空值 / 补空值测试	降级为阶段末；现在只改 RI 措辞

`updated` 改成「先读旧值再比较」	降级为阶段末；当前测试只要求新键 true
补 nil 键 / 空键 / 零值测试	降级为阶段末；当前测试没测
`checkRI` 实现并调用	降级为阶段末
补 Safety 的拷贝测试	降级为阶段末；现在只把 Safety 的句子写对

10.2.5. 修正后的下一步（只剩五件事）

顺序	动作	验收
1	删 `Set` 注释与 `func` 之间的空行	`go doc KV.Set` 显示正文
2	`Get` 的 nil 分支改成 `nil, false, nil`（与实现一致）	注释不再说「返回错误」
3	RI：删值约束、补 `mem != nil`；Safety 拆三句	`go doc KV` 里的 RI 能判真假
4	第 50 行 `xxx` → `k1`；`gofmt -w kv_test.go`	删除有验收；`gofmt -l` 无输出
5	把三个空 `TODO` 改成阶段末清单	没有空 TODO，且写明等什么

这五件全部只动注释和一行测试键，**不改实现、不新增测试**，符合「当前测试强度」原则。

10.2.6. 仍然建议现在做的一条：`Del` 的渲染

`Del` 的 Postconditions 用 tab 缩进，`go doc` 把它渲染成代码块而不是列表。这不影响测试，但影响你读 spec 的体验；而且改法只是把 tab 换成两空格。

10.3. 陷阱与易错

⚠ **陷阱预警「满足当前测试」不等于「注释可以随便写」。** 测试只验行为，注释是未来你（和客户）读的唯一合同。注释说假话，测试再绿也没用。

⚠ **陷阱预警 `TODO(阶段末)` 必须写触发条件。**否则它会变成「永远不做的 TODO」。写清「等作者给完本模块测试」，三个月后才知道何时该回来。

⚠ **陷阱预警 假绿不属于「测试强度」。** 第 50 行删了 `k1` 却查 `xxx`，这不是「作者还没给更强的测试」，而是现有测试写错了键。现在就该改。

⚠ **陷阱预警 不要为了「未来可能的测试」改实现。** 例如给 `Set` 加 `len(val)==0` 拒绝，可能让未来作者测试里合法的空值用例直接红掉。合同用 TODO 占位，实现按当前测试来。

10.4. 关键点

> 📌 **核心要点** 当前测试强度下，0101 的必做项只有五件：删 `Set` 注释空行、修 `Get` 的 nil 分支、 修 RI（删值约束 + 补 `mem != nil`）、改第 50 行的键、把空 TODO 换成阶段末清单。 其余全部推迟到作者测试补全之后。

事项	当前测试是否约束	处置
基本路径	是（7 条断言）	保持
未命中	是（`Get(xxx)`）	保持
nil / 空键	否	阶段末
nil / 空值	否	阶段末
同键同值	否	阶段末
生命周期	否	阶段末
注释与实现矛盾	否，但不属于「新测试」	现在修（改注释）
假绿	是（现有测试写错）	现在修

10.5. 变式检验

变式 1：如果未来的测试要求 `Set(nil)` 返回 `nil` 错误而不是 `ErrKeyIsNil`，怎么办？ 现在不改实现，把「nil 策略」留在阶段末清单里。届时以作者测试为准，一次改三个方法，不要现在猜。

变式 2：如果未来测试要求 `updated` 在同键同值时为 false 呢？ 当前实现会失败，届时再改「先读旧值再比较」。 现在只需在 spec 里写「成功路径返回 true；同键同值的行为待定」。

变式 3：RI 现在删掉值约束，未来测试要求拒绝空值怎么办？ 那时把约束加回来，并让 `Set` 拒绝空值、补测试。 RI 与实现必须同步改，不能只改注释。

变式 4：为什么第 50 行算「现在修」而不是「阶段末」？ 因为它不是「覆盖不足」，而是「断言写错了键」：删 `k1` 却查 `xxx`，即使删除逻辑完全失效也会通过。修它是让现有测试恢复有效性。

变式 5：`TODO(阶段末)` 和空 `TODO` 的区别是什么？ 空 `TODO` 只说明「这里缺东西」；`TODO(阶段末)` 还说明「什么时候回来、回来补什么」。 后者才是可执行的计划。

10.6. 同类归类

情形	处置	理由
测试未覆盖的行为	`TODO(阶段末)`	测试强度是唯一验收标准
测试已覆盖但注释不符	现在改注释	合同一致性，不扩大测试
测试本身写错	现在修测试	假绿不是强度问题
实现语义有争议	等作者测试	不猜未来接口

纯格式问题	现在修	零风险，改法唯一

10.7. Go 语言机制补充

测试是 Go 项目里的可执行规格。 `_test.go` 决定验收边界；`go test` 绿只说明当前断言满足，不说明合同完整。

`TODO` 不是编译器指令。 `go vet` 不检查它，`go test` 也不因它失败。它的价值是让「还没决定」在代码里显式可见；加不加 `（阶段末）` 全凭约定。

`gofmt -w` 会写回文件。只对格式做修改，不动语义；`gofmt -l` 检查、`gofmt -d` 看差异、`gofmt -w` 落地。

6.031 对照。 R04 的测试优先与「增量测试」正是你描述的模式：测试随功能逐步添加，规格随测试逐步加强。 但 R11 的 RI 要求仍然成立——任何时候，写下的 RI 都必须是当前实现的真话。

10.8. 本次操作与当前状态

已做的文件操作：

1. 读取 `internal/db/kv_test.go` 的 7 条断言，确认当前测试强度；
2. 对照第九节的 P0 / P1 清单，按「当前测试强度」重新分级；
3. 追加本一级标题到 `note/2026-11-12.typ`；
4. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. 本节为策略校准，未新增命令；依据是第九节的探针输出与 `kv_test.go` 的 7 条断言；
2. `typst compile 2026-11-12.typ` → 待编译确认（见下）。

需要你自己动手的：

1. 现在做（五件）：删 `Set` 注释空行、修 `Get` 的 nil 分支、修 RI、 第 50 行改 `k1`、空 TODO 改阶段末清单；
2. 顺手做：`Del` 的列表缩进、`gofmt -w kv_test.go`；
3. 阶段末（作者给完本模块测试后）：nil/空键、nil/空值、`updated` 同键同值、`Open` / `Close` 生命周期、零值、`checkRI`、Safety 测试。

Index of Listings

Listing 1 两种写法给出的都是可判真假的谓词 7
Listing 2 第一种写法的三段式版本：语义不变，只是把后置条件提到函数级 7
Listing 3 第二种写法的 JSDoc 版本：ReadonlyArray 让「不修改」从注释变成类型检查 8
Listing 4 标准库 bytes.ToLower：一句话同时给了后置条件（映射小写）和别名条款（copy） 8
Listing 5 标准库 slices.Sort：后置条件写了，但「就地修改 x」只能从实现看出 8
Listing 6 标准库 slices.Clone：连 nil 进 nil 出都写进了合同 9
Listing 7 GOROOT `src/io/io.go` 的 Reader 注释（节选） 14
Listing 8 GOROOT `src/sync/mutex.go`：零值可用 + 不得复制 14
Listing 9 `$TMPDIR` 下的最小复现：复制含 Mutex 的结构体 15
Listing 10 `go vet ./...` 的真实输出（退出码 1） 15
Listing 11 GOROOT `src/os/file.go`：关闭后不可用，重复调用返回错误 15
Listing 12 GOROOT `src/database/sql/sql.go`：幂等，且不影响 Err 的结果 15
Listing 13 GOROOT `src/net/http/client.go`（节选）：零值可用 + 可复用 + 并发安全 . 16
Listing 14 `google.golang.org/protobuf@v1.36.12/proto/decode.go`：写成前置条件 16
Listing 15 GOROOT `src/encoding/json/decode.go`：写成返回错误 16
Listing 16 `github.com/google/go-cmp@v0.7.0/cmp/compare.go`（节选：中间关于每行前缀的说明已略） 17
Listing 17 GOROOT `src/os/file.go`：错误类型属于合同 17
Listing 18 GOROOT `src/slices/sort.go`：写了结果，没写就地修改 17
Listing 19 GOROOT `src/slices/slices.go`（节选）：明说返回副本，连 nil 语义都写死 18
Listing 20 探针测试的核心片段（通过 `-overlay` 注入，未写入仓库） 24
Listing 21 GOROOT `src/encoding/json/decode.go`：stores the result 就是 effects 30
Listing 22 GOROOT `src/net/http/client.go` 的 `Client.Do`（节选）：资源副作用 + 失败语义 31
Listing 23 GOROOT `src/io/io.go`：反向副作用条款，约束实现者不得保留引用 31
Listing 24 GOROOT `src/slices/sort.go`：实际就地修改 x，注释没写 31
Listing 25 GOROOT `src/os/file.go`：改的是进程级 cwd，影响所有 goroutine 31
Listing 26 当前仓库的真实输出：只有签名，注释正文一个字都没有 36
Listing 27 `go doc` 的真实渲染：三个子项被并进父项段落，`-val` / `-ok` / `-err` 成了普通文本 37
Listing 28 输入：两空格缩进 + `*` 标记 39
Listing 29 gofmt 输出：`*` 被换成 `-`，缩进保持两空格 39
Listing 30 gofmt 统一有序列表标记为 `1.` 39
Listing 31 gofmt 输出：代码块每行以一个 tab 开头（`//` 后紧跟制表符） 39
Listing 32 条件用段落、结论用一级列表：三段都合法，渲染不会合并 41
Listing 33 当前基线：测试绿，但 `kv_test.go` 还没过 gofmt 45
Listing 34 当前输出只有签名：注释与 `func` 之间又有一个空行 50

Listing 35 Set 的实际行为（`go test -overlay=... -run TestProbeSetSpec -v`） 50
Listing 36 spec 声称空切片键返回 `(false, nil)` 51
Listing 37 同一个键（空串），在 Set 里被区别对待，在 Get 里又不区分 51
Listing 38 写后比较：`newVal` 是 `val` 的克隆，`bytes.Equal` 必然为 true 52
Listing 39 列表用 tab 缩进；去掉空行后 gofmt 会把它改成两空格 + `-` 52
Listing 40 `json.Unmarshal`：第一句是 effect（stores the result），第二句是 postcondition（返回什么错误） 58
Listing 41 `bytes.ToLower`：只有 postcondition，没有 effect——它不改 `s` 58
Listing 42 `os.File.Close`：effect（关闭、取消 I/O）与 postcondition（重复调用返回错误）分成两句 58
Listing 43 Del / Set / Get 的边界行为实测（Go 1.27，本机） 61
Listing 44 注释与 `func` 之间的空行：`go doc KV.Set` 只输出签名 62
Listing 45 当前 RI：键约束现在由 Set 维护，值约束完全没维护 63
Listing 46 存 nil 值后 Get 命中，说明 RI 的值约束没有被任何路径维护 63
Listing 47 当前 Safety：方向对，但 `mem.val` 不是合法表达，也没有分向 63
Listing 48 当前测试的七条断言：只覆盖基本路径与「未命中」 68
Listing 49 阶段末清单：写清「等什么」和「补什么」，而不是一个空 TODO 69